

# Python APIs

## Learn the contract. Write clear Python. Practice one concept at a time.

A visual, hands-on course with FastAPI and HTTPX.

| 16 focused modules | 48 small exercises     | 150 review questions                   |
|--------------------|------------------------|----------------------------------------|
| HTTP to containers | Three tasks per module | 50 open response + 100 multiple choice |

For Python users who want to understand both consuming and building APIs. Includes explicit function, class and module patterns, deterministic local examples, and answers separated from the final assessments.

### Your learning loop



English edition | Prepared 5 September 2026

Recommended pace: 45-75 minutes per module, plus review.

# How to use this course

Prerequisites: basic Python functions, classes, imports, dictionaries, exceptions and terminal commands. You do not need frontend development, an external database, a cloud account or prior web-framework experience.

## Keep every exercise isolated

Create a new folder for each module: lab\_01, lab\_02, and so on. Each worked example starts from the files printed in that module. Reuse only the Python environment. Stop the previous server before starting another on port 8000. Module 06 and later multi-file examples name each file explicitly.

## Three pages per module

| Page              | How to work                                                                                               |
|-------------------|-----------------------------------------------------------------------------------------------------------|
| Concept           | Read the explanation, follow the arrows and inspect the comparison.                                       |
| Worked example    | Copy the complete file(s), run the command and check the stated output.                                   |
| Isolated practice | Complete three small tasks. Read the solution sketch only after attempting them; check the exit criteria. |

## What the code labels mean

A worked example is complete for its named files and the shared environment. A solution sketch is an insertion, replacement or test fragment: its comments identify the context. Ellipses are used only for a Protocol method contract. No database or network account is hidden behind a placeholder.

## Scope and design choices

FastAPI is the teaching framework and HTTPX is the client. The design habits transfer to other frameworks. There is no universal best class hierarchy: the examples use the smallest clear boundary that supports validation, substitution and resource ownership.

Module 04 previews updates without saving them. Module 11 uses a public demo token. Module 15 uses a fake model. Module 16 needs Docker. These limits keep each lesson focused and are stated where they matter.

## Assessment method

Finish the 50 open-response questions and 100 single-answer multiple-choice questions before opening the answer sections. Work in short sessions. For open questions, score each 0-4: correct core idea (2), reasoning or valid code (1), and the requested boundary or edge case (1). MC questions score 1 each, with no penalty. These are self-study indicators, not certification thresholds.

# One setup, separate labs

Use Python 3.12 for the tested course baseline. The commands below target macOS/Linux. Create the environment once, then activate it whenever you return to the course.

```
mkdir python-api-course
cd python-api-course
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install "fastapi==0.115.12" "httpx==0.28.1" \
    "pydantic==2.11.7" "uvicorn==0.34.3" "pytest==8.4.1"
mkdir lab_01
cd lab_01
```

These are deliberate course versions, not a claim about the newest releases. Keep the baseline while learning; upgrade in a separate environment and rerun the checks. A fresh virtual environment prevents unrelated project packages entering the lock file.

## For each module

```
mkdir -p ../lab_02
cd ../lab_02
# Copy the module example into its named file(s).
python -m uvicorn main:app --reload
```

## Send JSON from another terminal

```
curl -i -X POST "http://127.0.0.1:8000/quotes/12?discount=10" \
  -H "Content-Type: application/json" \
  -d '{"quantity":2}'
```

The curl example applies to Module 03. For other labs, replace only the path and body. -i includes response headers; -X sets the method; -H adds a header; -d sends the body. Swagger UI at /docs is an alternative for manual calls.

## Common setup fixes

| Symptom                 | Targeted fix                                                                   |
|-------------------------|--------------------------------------------------------------------------------|
| No module named fastapi | Activate .venv; install with the same python -m pip.                           |
| Could not import main   | Run from the folder that contains main.py.                                     |
| Address already in use  | Stop the previous server or choose --port 8001.                                |
| Windows shell           | Activate with .venv\Scripts\Activate.ps1; use curl.exe and PowerShell quoting. |

# Course map

| Module | Focus                                         | Pages |
|--------|-----------------------------------------------|-------|
| 01     | Read an HTTP exchange                         | 5-7   |
| 02     | Build one FastAPI endpoint                    | 8-10  |
| 03     | Validate path, query and body                 | 11-13 |
| 04     | Design resources and partial updates          | 14-16 |
| 05     | Return useful errors                          | 17-19 |
| 06     | Choose functions, classes and modules         | 20-22 |
| 07     | Inject dependencies explicitly                | 23-25 |
| 08     | Test observable API behavior                  | 26-28 |
| 09     | Understand async and blocking work            | 29-31 |
| 10     | Call another API reliably                     | 32-34 |
| 11     | Authenticate and authorize                    | 35-37 |
| 12     | Paginate and evolve a contract                | 38-40 |
| 13     | Observe requests and understand CORS          | 41-43 |
| 14     | Persist one resource with SQLite              | 44-46 |
| 15     | Manage application lifetime and configuration | 47-49 |
| 16     | Run the API in a container                    | 50-52 |

| Reference and assessment                 | Start page |
|------------------------------------------|------------|
| HTTP and design desk references          | 53         |
| 50 open-response questions               | 55         |
| 100 multiple-choice questions            | 65         |
| Open-response model answers              | 85         |
| Multiple-choice answers and explanations | 95         |
| Official documentation links             | 100        |

Suggested sequence: Modules 01-04 establish the contract; 05-08 build maintainable code; 09-13 handle boundaries; 14-16 cover persistence, lifetime and packaging. All example folders remain independent.

# Read an HTTP exchange

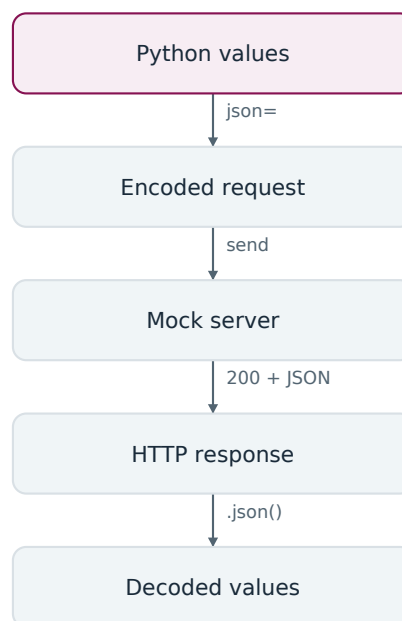
## Identify what crosses the network before writing a server.

An API is a contract that lets one program ask another for an operation or data. This course focuses on HTTP APIs. A Python function call stays inside a process; an HTTP request crosses a boundary where latency, disconnections and independent failures become possible.

A request has a method, a target URL, headers and sometimes a body. In `https://api.example.test/items/7?verbose=true`, HTTPS is the scheme, `api.example.test` is the host, `/items/7` is the path and `verbose=true` is a query parameter. A response has a status code, headers and possibly a body.

JSON is a text format, not a Python dictionary. JSON uses double-quoted keys, `true/false` and `null`. HTTPX encodes a Python dictionary when you pass `json=`; `response.json()` decodes JSON back into Python values. Parsing succeeds only if the body is valid JSON.

The mock below is a local stand-in for a network server. It teaches the request/response boundary without an account, internet connection or running API. It does not test DNS, TLS or real network behavior.



| Element      | Example          | Meaning                   |
|--------------|------------------|---------------------------|
| Method       | GET              | Read a representation     |
| Accept       | application/json | Preferred response format |
| Content-Type | application/json | Format of the actual body |
| Status       | 200              | Request succeeded         |

References: [01], [02]. Linked reading appears at the end.

# Read, run, inspect

## FILES main.py

```
import httpx

def mock_server(request: httpx.Request) -> httpx.Response:
    assert request.method == "GET"
    assert request.url.params["unit"] == "c"
    return httpx.Response(200, json={"temperature": 28})

def main() -> None:
    transport = httpx.MockTransport(mock_server)
    with httpx.Client(transport=transport) as client:
        response = client.get(
            "https://weather.example.test/current",
            params={"unit": "c"},
            headers={"Accept": "application/json"},
        )
        response.raise_for_status()
        print(response.status_code)
        print(response.json())

if __name__ == "__main__":
    main()
```

## Run from this lab folder

```
python main.py
```

Output: 200, then a Python dictionary containing temperature: 28. The .test domain is illustrative; MockTransport intercepts it.

The next page changes only this local example. No earlier lab service is required.

# Change one thing at a time

Suggested time: 20-35 minutes. Attempt the tasks before reading the sketch.

## Task 01.1

Change the query parameter to unit=f. Update the mock assertion and return 82.

## Task 01.2

Print `response.headers["content-type"]`. Explain why it describes the response, not the request.

## Task 01.3

Return `text="not JSON"` from the mock. Observe and name the failure at `response.json()`.

## Hint

Keep the transport. Only edit one parameter, one mock value or one response format at a time.

## Solution sketch

```
# Replace only the return statement in mock_server:
return httpx.Response(200, json={"temperature": 82})

# Add after response.raise_for_status():
print(response.headers["content-type"])
# Expect application/json for the JSON response.
```

## Exit criteria

- You can label method, URL, query, headers, status and body.
- You distinguish HTTP success from successful JSON parsing.

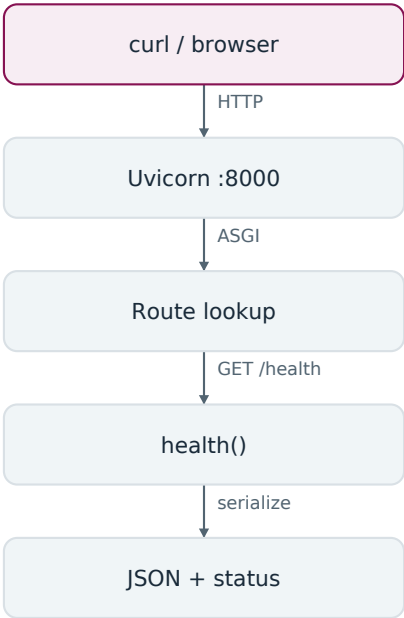
# Build one FastAPI endpoint

## Run a server and inspect its HTTP contract.

FastAPI is the application framework; Uvicorn is the server process that receives HTTP and invokes the application through ASGI. Your app object describes routes. A route combines a method and a path, so GET /health and POST /health are different operations.

A decorator registers a handler function. A plain def handler is appropriate here. Keep the handler small: receive validated input, call application logic and return an explicit response shape. A response model documents and checks that shape.

127.0.0.1 is the local loopback address. Port 8000 selects the listening service. The import string main:app means import the module main and use its variable app; it is not a URL or a filesystem path. /docs is an interactive view of the generated OpenAPI contract. /openapi.json is the machine-readable schema. Inspect both, but also use curl: documentation alone does not prove runtime behavior.



| Address or command           | Purpose                                |
|------------------------------|----------------------------------------|
| http://127.0.0.1:8000/health | Call the operation                     |
| http://127.0.0.1:8000/docs   | Inspect and try the contract           |
| main:app                     | Python module and app variable         |
| --reload                     | Restart after edits during development |

References: [03]. Linked reading appears at the end.

# Read, run, inspect

**FILES** main.py

```
from typing import Literal

from fastapi import FastAPI
from pydantic import BaseModel

class HealthResponse(BaseModel):
    status: Literal["ok"]

def create_app() -> FastAPI:
    application = FastAPI(title="API Lab", version="1.0.0")

    @application.get("/health", response_model=HealthResponse)
    def health() -> HealthResponse:
        return HealthResponse(status="ok")

    return application

app = create_app()
```

## Run from this lab folder

```
python -m uvicorn main:app --reload
```

In another terminal: `curl -i http://127.0.0.1:8000/health`. Expect 200 and `{"status":"ok"}`. Ctrl+C stops the server.

The next page changes only this local example. No earlier lab service is required.

# Change one thing at a time

Suggested time: 20-35 minutes. Attempt the tasks before reading the sketch.

## Task 02.1

Add GET /version returning {"version":"1.0.0"}, with a separate response model.

## Task 02.2

Call an unknown path and record the status. Call POST /health with curl -X POST.

## Task 02.3

Change the declared health value to an invalid literal in a copy of the example and observe where validation fails.

## Hint

Add the new class at module level and the new route inside create\_app(), before return application.

## Solution sketch

```
class VersionResponse(BaseModel):
    version: str

# Inside create_app(), before returning application:
@app.application.get("/version", response_model=VersionResponse)
def version() -> VersionResponse:
    return VersionResponse(version="1.0.0")
```

## Exit criteria

- GET /version returns 200 with a version string.
- Unknown path returns 404; POST /health returns 405.
- A server-created invalid response is a server defect, not a client 422.

# Validate path, query and body

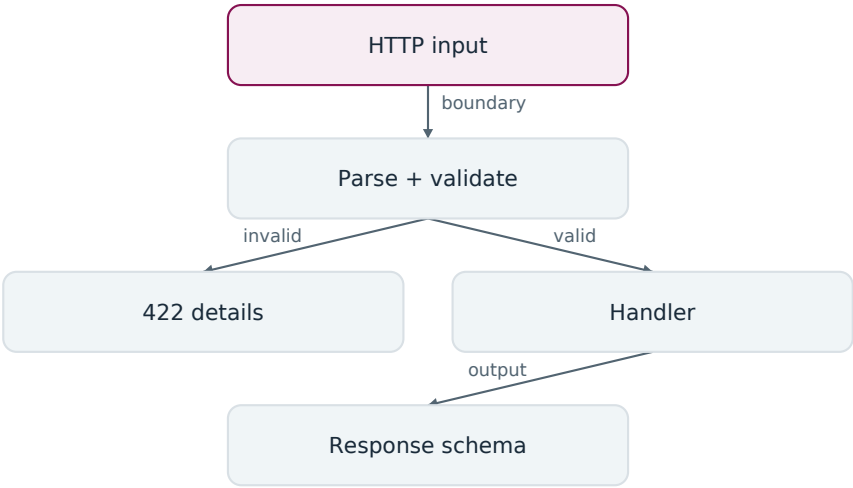
**Put input rules at the API boundary.**

A path parameter identifies the target; query parameters refine a request; a JSON body carries structured input. Choose location by meaning, not by what is easiest to parse. Credentials belong in the authentication mechanism, not a query string.

Python annotations document expected types. Pydantic performs runtime parsing and validation at the boundary. Parsing may coerce compatible input, so a type annotation alone is not a promise of strict input types. Use `strict=True` where coercion would be wrong.

Constraints express local rules: positive identifiers, bounded quantities and nonempty strings. Business rules that depend on state, such as whether an account may order, belong in application logic. Validation cannot prove authorization.

FastAPI normally returns 422 for request validation failures. The handler does not run for rejected input. Keep request and response models separate so internal fields cannot accidentally become part of the public contract.



| Input            | Declaration                     | Example                      |
|------------------|---------------------------------|------------------------------|
| Path             | <code>Path(gt=0)</code>         | <code>/quotes/12</code>      |
| Query            | <code>Query(ge=0, le=50)</code> | <code>?discount=10</code>    |
| Body             | <code>BaseModel + Field</code>  | <code>{"quantity": 2}</code> |
| Unknown body key | <code>extra="forbid"</code>     | Reject misspelled fields     |

References: [04], [05]. Linked reading appears at the end.

# Read, run, inspect

## FILES main.py

```
from typing import Annotated

from fastapi import FastAPI, Path, Query
from pydantic import BaseModel, ConfigDict, Field

app = FastAPI()

class QuoteRequest(BaseModel):
    model_config = ConfigDict(extra="forbid")
    quantity: int = Field(ge=1, le=100, strict=True)

class QuoteResponse(BaseModel):
    item_id: int
    total_cents: int

@app.post("/quotes/{item_id}", response_model=QuoteResponse)
def quote(
    item_id: Annotated[int, Path(gt=0)],
    payload: QuoteRequest,
    discount: Annotated[int, Query(ge=0, le=50)] = 0,
) -> QuoteResponse:
    unit_price_cents = 100
    total = payload.quantity * unit_price_cents
    return QuoteResponse(
        item_id=item_id,
        total_cents=total * (100 - discount) // 100,
    )
```

## Run from this lab folder

```
python -m uvicorn main:app --reload
```

POST /quotes/12?discount=10 with {"quantity":2} returns {"item\_id":12,"total\_cents":180}. Use /docs or the curl pattern in the setup pages.

The next page changes only this local example. No earlier lab service is required.

# Change one thing at a time

Suggested time: 20-35 minutes. Attempt the tasks before reading the sketch.

## Task 03.1

Try quantities 0, 1, 100 and 101. Record the two valid boundaries.

## Task 03.2

Try quantity="2" as a JSON string. Explain why strict=True rejects it.

## Task 03.3

Add a body field note: str with a maximum length of 40 and default "". Test 41 characters.

## Hint

Change only QuoteRequest for the new field. You do not need storage or another endpoint.

## Solution sketch

```
# Inside QuoteRequest:
note: str = Field(default="", max_length=40)

# Valid JSON body:
# {"quantity": 2, "note": "deliver before noon"}
# No new route is needed.
```

## Exit criteria

- Quantity 1 and 100 succeed; 0 and 101 return 422.
- Negative item\_id, discount=51 and extra body keys return 422.
- The 41-character note fails before the handler runs.

# Design resources and partial updates

## Distinguish method semantics from endpoint implementation.

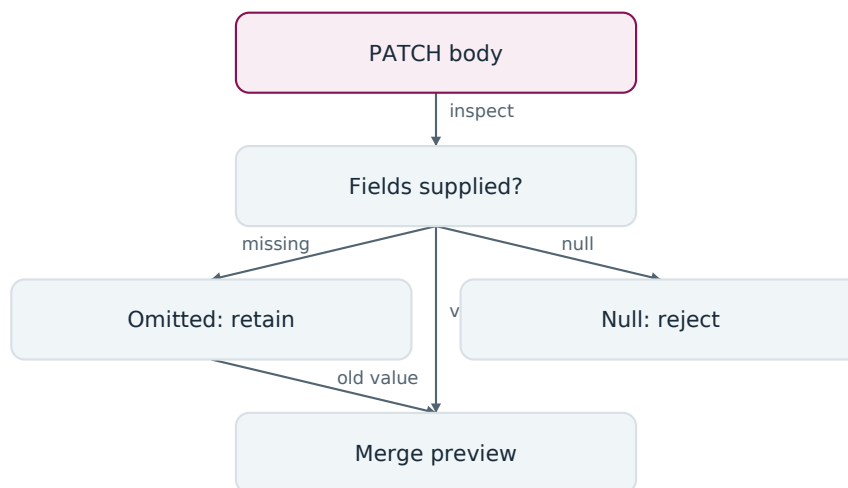
Use nouns for resources, such as /items, and HTTP methods for operations. GET retrieves; POST commonly creates a subordinate resource; PUT represents replacement at a known target; PATCH applies a partial update; DELETE removes the target association.

Safe means the client is not asking for a state change. Idempotent means repeating the same request has the same intended server-side effect as sending it once. It does not require identical response codes. A second DELETE can return 404 after the first succeeds.

A partial update must distinguish an omitted field from an explicit null.

`model_dump(exclude_unset=True)` keeps fields the client supplied. If null is forbidden, reject it deliberately; do not silently treat it as omission.

This lab computes an update preview and intentionally does not persist it. The seed row is recreated per request, making the example independent and repeatable. Module 14 introduces actual persistence.



| Method | Safe | Idempotent by semantics        |
|--------|------|--------------------------------|
| GET    | Yes  | Yes                            |
| POST   | No   | No general guarantee           |
| PUT    | No   | Yes                            |
| PATCH  | No   | Depends on the patch operation |
| DELETE | No   | Yes                            |

References: [06], [07]. Linked reading appears at the end.

# Read, run, inspect

## FILES main.py

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, ConfigDict, Field

app = FastAPI()

class ItemPatch(BaseModel):
    model_config = ConfigDict(extra="forbid")
    name: str | None = Field(default=None, min_length=1)
    active: bool | None = None

class ItemResponse(BaseModel):
    id: int
    name: str
    active: bool

@app.patch("/items/{item_id}", response_model=ItemResponse)
def preview_update(item_id: int, payload: ItemPatch) -> ItemResponse:
    if item_id != 1:
        raise HTTPException(404, detail="Item not found")
    changes = payload.model_dump(exclude_unset=True)
    if any(value is None for value in changes.values()):
        raise HTTPException(422, detail="Null values are not allowed")
    current = {"id": 1, "name": "Pen", "active": True}
    return ItemResponse.model_validate(current | changes)
```

## Run from this lab folder

```
python -m uvicorn main:app --reload
```

PATCH /items/1 with {"active":false} returns id=1, name="Pen", active=false. The next request starts from the seed again.

The next page changes only this local example. No earlier lab service is required.

# Change one thing at a time

Suggested time: 20-35 minutes. Attempt the tasks before reading the sketch.

## Task 04.1

Send {}, {"name":"Pencil"}, and {"name":null}; explain each result.

## Task 04.2

On paper design POST /items, GET /items/{id}, PUT /items/{id} and DELETE /items/{id}; state success codes and missing-resource behavior.

## Task 04.3

Write a pure function that replaces both mutable fields from a complete input model. Do not add storage.

## Hint

PUT input should require all fields your replacement contract considers writable. The identifier can stay in the path.

## Solution sketch

```
class ItemReplace(BaseModel):
    name: str = Field(min_length=1)
    active: bool

def replace_item(item_id: int, payload: ItemReplace) -> ItemResponse:
    return ItemResponse(id=item_id, **payload.model_dump())
```

## Exit criteria

- Empty PATCH retains both fields; explicit null returns 422.
- You explain why increment-by-one PATCH is not idempotent.
- Your DELETE 204 design sends no JSON body.

# Return useful errors

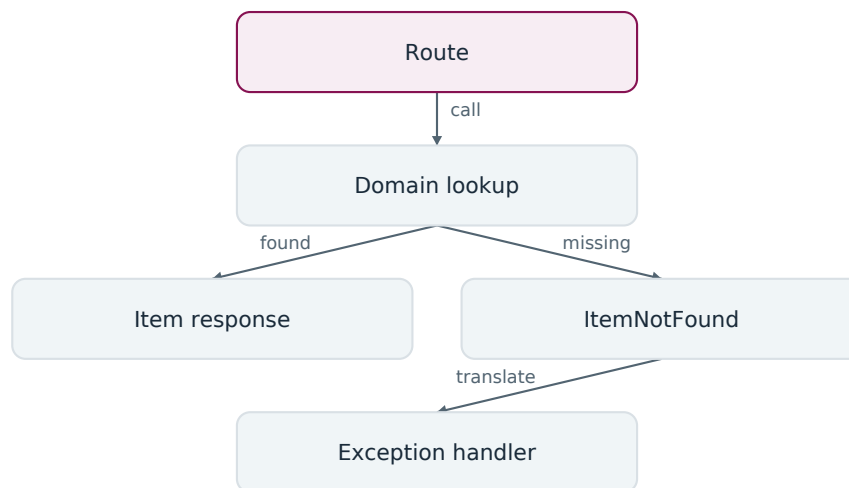
## Separate expected domain failures from unexpected defects.

An API error needs a useful status and a stable machine-readable meaning. Human text may evolve; clients should branch on a code such as `item_not_found` rather than matching an English sentence. Do not return 200 with an error disguised as successful data.

A domain function should not need HTTP to express a missing item. Raise a domain exception there, then translate it at the web boundary. This keeps the same function usable from a command-line job or a unit test.

Use 404 for an unavailable resource, 409 for a state conflict and 422 for invalid input under this course contract. Unexpected defects belong in 500-class responses with internal logging, without returning stack traces or secrets to callers.

The example defines one error envelope for its domain exception. FastAPI validation and other HTTP errors keep their default envelope. A single envelope for every error requires additional handlers; do not claim consistency that you have not implemented.



| Failure           | HTTP mapping | Client reaction                   |
|-------------------|--------------|-----------------------------------|
| Invalid request   | 422          | Correct the input                 |
| Missing item      | 404          | Correct ID or show absence        |
| State conflict    | 409          | Resolve conflict first            |
| Unexpected defect | 500          | Report; avoid blind write retries |

References: [08]. Linked reading appears at the end.

# Read, run, inspect

## FILES main.py

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

app = FastAPI()

class ItemNotFound(Exception):
    pass

def find_item(item_id: int) -> dict[str, int | str]:
    if item_id != 1:
        raise ItemNotFound()
    return {"id": 1, "name": "Pen"}

@app.exception_handler(ItemNotFound)
async def item_not_found_handler(
    request: Request, exc: ItemNotFound,
) -> JSONResponse:
    return JSONResponse(
        status_code=404,
        content={"error": {
            "code": "item_not_found",
            "message": "The requested item does not exist.",
        }},
    )

@app.get("/items/{item_id}")
def get_item(item_id: int) -> dict[str, int | str]:
    return find_item(item_id)
```

## Run from this lab folder

```
python -m uvicorn main:app --reload
```

GET /items/2 returns 404 and error.code=item\_not\_found. GET /items/not-an-int returns the default 422 validation body.

The next page changes only this local example. No earlier lab service is required.

# Change one thing at a time

Suggested time: 20-35 minutes. Attempt the tasks before reading the sketch.

## Task 05.1

Introduce a domain exception `ItemConflict` and map it to 409 with code `item_conflict`.

## Task 05.2

Trigger it for `item_id=3` in `find_item`, before the missing-item check.

## Task 05.3

Explain why catching Exception and returning an empty list would hide a defect.

## Hint

Duplicate the narrow handler structure, not a broad catch-all. Keep domain code free of FastAPI imports in a larger app.

## Solution sketch

```
class ItemConflict(Exception):
    pass

@app.exception_handler(ItemConflict)
async def conflict_handler(
    request: Request, exc: ItemConflict,
) -> JSONResponse:
    return JSONResponse(
        status_code=409,
        content={"error": {"code": "item_conflict"}},
    )

# At the start of find_item:
# if item_id == 3:
#     raise ItemConflict()
```

## Exit criteria

- ID 1 stays successful; ID 2 stays 404; ID 3 becomes 409.
- The HTTP handler owns translation; the domain owns meaning.

# Choose functions, classes and modules

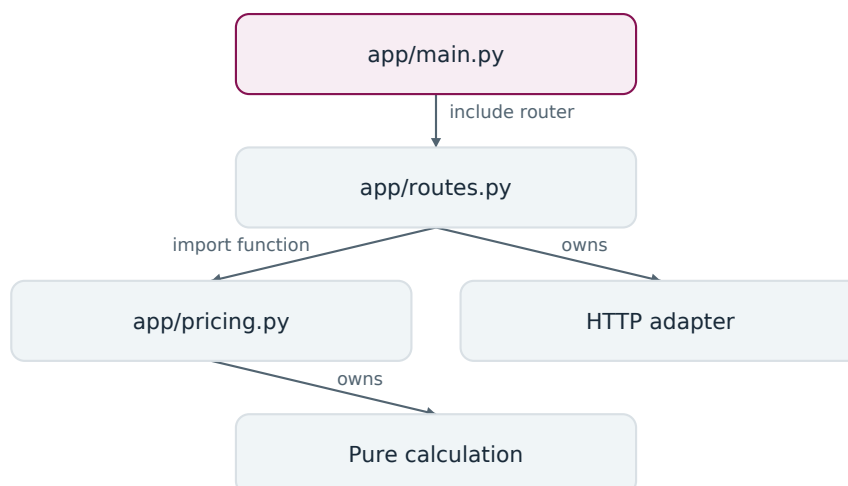
## Organize code by responsibility without creating empty layers.

Use a function for a stateless calculation. Use a class when an object has dependencies, configuration or state with a meaningful lifetime. Use a module to group one coherent responsibility. A class containing only unrelated static methods adds little clarity.

A route owns HTTP concerns, a service owns a use case, and a repository owns persistence operations. These are roles, not a requirement to create three layers for a ten-line endpoint. Add a seam when there is a concrete need to swap, test or reuse behavior.

This example uses a pure pricing function and a router. The app factory assembles them. Imports flow from the composition root toward the feature; pricing.py does not import main.py. That direction prevents circular dependencies.

All files below live in one fresh lab directory. `app/__init__.py` is an empty file that marks the package. Run Uvicorn from the parent directory of `app`, not from inside the package.



| Construct   | Use it for                            | Avoid                         |
|-------------|---------------------------------------|-------------------------------|
| Function    | Deterministic transformation          | Hidden global state           |
| Class       | A dependency or stateful collaborator | A namespace of static methods |
| Module      | A coherent set of behavior            | A giant <code>utils.py</code> |
| App factory | Dependency assembly                   | Business rules                |

References: [09]. Linked reading appears at the end.

# Read, run, inspect

**FILES** Four files, separated by filename comments

```
# app/pricing.py
def calculate_total(quantity: int, unit_cents: int) -> int:
    if quantity < 1 or unit_cents < 0:
        raise ValueError("Invalid pricing input")
    return quantity * unit_cents

# app/routes.py
from typing import Annotated
from fastapi import APIRouter, Query
from app.pricing import calculate_total

router = APIRouter(prefix="/prices", tags=["prices"])

@router.get("/total")
def total(quantity: Annotated[int, Query(ge=1)] = 1) -> dict[str, int]:
    return {"total_cents": calculate_total(quantity, 125)}

# app/main.py
from fastapi import FastAPI
from app.routes import router

def create_app() -> FastAPI:
    application = FastAPI()
    application.include_router(router)
    return application

app = create_app()

# app/__init__.py: empty file
```

## Run from this lab folder

```
python -m uvicorn app.main:app --reload
```

GET /prices/total?quantity=2 returns {"total\_cents":250}. The pricing function can also be called without starting a server.

Multi-file examples: split at the filename comments. Do not paste non-Python files into a Python module.

# Change one thing at a time

Suggested time: 20-35 minutes. Attempt the tasks before reading the sketch.

## Task 06.1

Add a pure `calculate_discount(total_cents, percent)` function in `pricing.py`. Accept percent from 0 through 100.

## Task 06.2

Call the function directly with 250 and 10; expect 225.

## Task 06.3

Explain when a `PricingService` class would become useful, for example when it receives a tax-rate provider.

## Hint

Choose and document a rounding policy. For this integer-cents exercise, round down using integer division.

## Solution sketch

```
# Add to app/pricing.py:
def calculate_discount(total_cents: int, percent: int) -> int:
    if total_cents < 0 or not 0 <= percent <= 100:
        raise ValueError("Invalid discount input")
    return total_cents * (100 - percent) // 100
```

## Exit criteria

- The function imports no web framework.
- Zero and 100 percent work; -1 and 101 raise `ValueError`.
- No module imports the app factory to access business logic.

# Inject dependencies explicitly

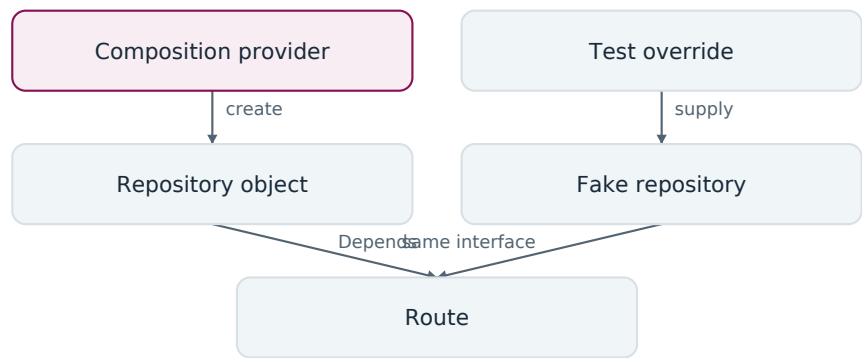
## Swap a collaborator without changing a route.

Dependency injection means a function receives what it needs instead of constructing everything internally. In FastAPI, Depends declares a provider. The provider is a normal callable, which makes dependency creation visible and replaceable.

A Protocol defines the operations a collaborator must expose. It describes behavior structurally; a fake does not have to inherit from it. Use this seam when multiple implementations or test doubles are useful, not for every small function.

Dependency caching is normally scoped to a request, not to the whole application. A provider that constructs an object is not automatically a global singleton. Shared clients and models need an intentional lifetime, introduced in Module 15.

This lab uses a repository that always returns fixed data. There is no database, mutable global collection or repository setup. Its only purpose is to show a dependency boundary.



| Piece               | Responsibility                         |
|---------------------|----------------------------------------|
| Protocol            | Describe the needed operation          |
| Concrete repository | Implement that operation               |
| Provider function   | Choose or construct the implementation |
| Route parameter     | Receive the dependency explicitly      |

References: [10], [11]. Linked reading appears at the end.

# Read, run, inspect

## FILES main.py

```
from typing import Annotated, Protocol
from fastapi import Depends, FastAPI

app = FastAPI()

class ItemRepository(Protocol):
    def count(self) -> int: ...

class FixedItemRepository:
    def count(self) -> int:
        return 3

def get_repository() -> ItemRepository:
    return FixedItemRepository()

RepositoryDep = Annotated[ItemRepository, Depends(get_repository)]

@app.get("/items/count")
def count_items(repository: RepositoryDep) -> dict[str, int]:
    return {"count": repository.count()}
```

## Run from this lab folder

```
python -m uvicorn main:app --reload
```

GET /items/count returns {"count":3}. The route does not know which concrete repository supplies the count.

The next page changes only this local example. No earlier lab service is required.

# Change one thing at a time

Suggested time: 20-35 minutes. Attempt the tasks before reading the sketch.

## Task 07.1

Create a fake repository that returns 9.

## Task 07.2

Override `get_repository` using `app.dependency_overrides` in a `TestClient` context. Assert `count=9`.

## Task 07.3

Clear overrides in finally so the change cannot leak into another test.

## Hint

Override with a callable that returns an object, not with the object itself.

## Solution sketch

```
from fastapi.testclient import TestClient

class FakeRepository:
    def count(self) -> int:
        return 9

app.dependency_overrides[get_repository] = lambda: FakeRepository()
try:
    with TestClient(app) as client:
        assert client.get("/items/count").json() == {"count": 9}
finally:
    app.dependency_overrides.clear()
```

## Exit criteria

- The fake works without inheriting from the Protocol.
- No route code changes; the override is removed afterward.

# Test observable API behavior

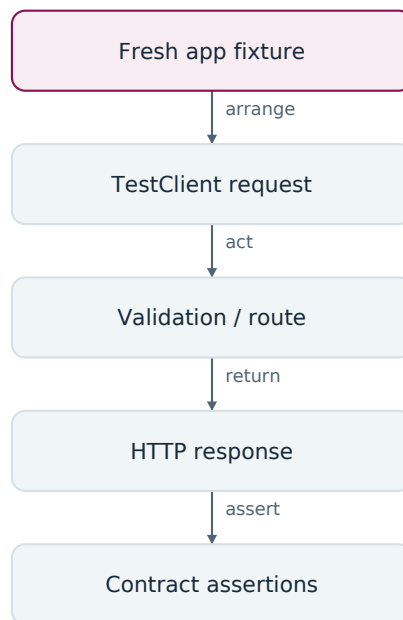
## Write small tests that verify the public contract.

A unit test targets pure logic. An API test sends an HTTP-shaped request through the application and checks status, body or headers. TestClient runs in process, so it does not prove that a deployed port, proxy or TLS certificate works.

Use arrange, act, assert: build a clean app, send one meaningful request, then check the contract. Prefer a fresh app fixture to shared mutable test state. A context manager also runs the application lifespan when one exists.

Test boundaries and failure paths, not only a happy example. For a positive integer field, useful cases include the smallest valid value, zero and a wrong type. Avoid asserting volatile details such as the complete wording of every validation error.

These tests live beside their application in a fresh directory. Running `python -m pytest -q` from that directory keeps imports predictable and requires no Uvicorn process.



| Layer       | What to verify             | Example                                   |
|-------------|----------------------------|-------------------------------------------|
| Unit        | Business calculation       | $2 \times 125 = 250$                      |
| API         | Input/output contract      | Invalid input returns 422                 |
| Integration | Real collaborator behavior | SQLite commit persists                    |
| Smoke       | Running service reachable  | <code>curl /health</code> on deployed URL |

References: [12]. Linked reading appears at the end.

# Read, run, inspect

## FILES main.py + test\_main.py

```
# main.py
from fastapi import FastAPI
from pydantic import BaseModel, Field

class DoubleRequest(BaseModel):
    value: int = Field(ge=1, strict=True)

def create_app() -> FastAPI:
    app = FastAPI()

    @app.post("/double")
    def double(payload: DoubleRequest) -> dict[str, int]:
        return {"result": payload.value * 2}

    return app

# test_main.py
from collections.abc import Iterator
import pytest
from fastapi.testclient import TestClient
from main import create_app

@pytest.fixture
def client() -> Iterator[TestClient]:
    with TestClient(create_app()) as test_client:
        yield test_client

def test_double_success(client: TestClient) -> None:
    response = client.post("/double", json={"value": 2})
    assert response.status_code == 200
    assert response.json() == {"result": 4}

@pytest.mark.parametrize("value", [0, -1, "2"])
def test_double_rejects_invalid(
    client: TestClient, value: int | str,
) -> None:
    response = client.post("/double", json={"value": value})
    assert response.status_code == 422
```

## Run from this lab folder

```
python -m pytest -q
```

Expect four passing cases: one successful request plus three parameterized invalid inputs.

Multi-file examples: split at the filename comments. Do not paste non-Python files into a Python module.

# Change one thing at a time

Suggested time: 20-35 minutes. Attempt the tasks before reading the sketch.

## Task 08.1

Add a test for the boundary value 1.

## Task 08.2

Add a test for a missing body field using `json={}.`

## Task 08.3

Change the handler to multiply by 3 temporarily; identify which test detects the regression and restore it.

## Hint

Assert the externally visible result. You do not need to inspect implementation calls or mock multiplication.

## Solution sketch

```
def test_minimum_value(client: TestClient) -> None:
    response = client.post("/double", json={"value": 1})
    assert response.status_code == 200
    assert response.json() == {"result": 2}

def test_missing_value(client: TestClient) -> None:
    response = client.post("/double", json={})
    assert response.status_code == 422
```

## Exit criteria

- Six cases pass after the two additions.
- The suite runs without an external server or network.

# Understand async and blocking work

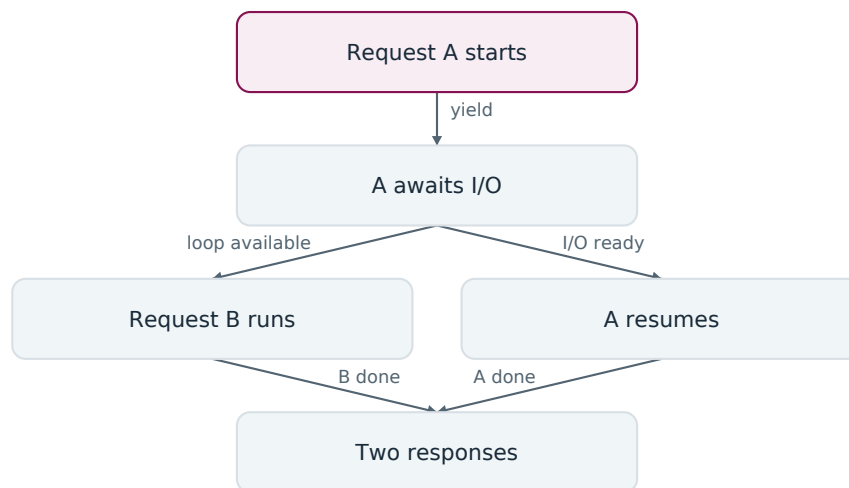
## Choose execution style based on the work being performed.

Concurrency lets requests make progress while other requests wait. Parallelism performs work simultaneously. `async def` does not make a CPU calculation faster; it allows cooperative suspension at await points when the awaited operation supports it.

In FastAPI, ordinary `def` route handlers run in a thread pool. An `async def` handler runs on the event loop. Calling a blocking library directly inside it can stall other tasks. A helper called directly is not automatically moved to a thread merely because it is defined with `def`.

Use `async def` with awaitable I/O clients. Use `def` for simple routes that rely on blocking I/O, or deliberately offload a blocking call. Threads are finite resources. Heavy CPU or GPU workloads need separate capacity planning rather than an `async` keyword.

The sleep in this lab simulates I/O waiting; it is not an HTTP timeout or a performance benchmark. Run two local requests together to observe overlap. Real latency also includes queuing, network, serialization and downstream work.



| Work in handler        | Pattern                                       |
|------------------------|-----------------------------------------------|
| Awaitable network I/O  | <code>async def</code> + <code>await</code>   |
| Blocking I/O library   | <code>def</code> , or explicit thread offload |
| Short pure calculation | Either; keep it short                         |
| Heavy CPU inference    | Separate worker or serving capacity           |

References: [13]. Linked reading appears at the end.

# Read, run, inspect

## FILES main.py + client.py

```
import asyncio
from typing import Annotated
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/wait")
async def wait_for_result(
    seconds: Annotated[float, Query(ge=0, le=2)] = 0.2,
) -> dict[str, float]:
    await asyncio.sleep(seconds)
    return {"waited": seconds}

# client.py (a separate file)
import asyncio
from time import perf_counter
import httpx

async def main() -> None:
    async with httpx.AsyncClient(base_url="http://127.0.0.1:8000") as c:
        start = perf_counter()
        responses = await asyncio.gather(
            c.get("/wait", params={"seconds": 0.5}),
            c.get("/wait", params={"seconds": 0.5}),
        )
        for response in responses:
            response.raise_for_status()
        print(round(perf_counter() - start, 2))

if __name__ == "__main__":
    asyncio.run(main())
```

## Run from this lab folder

```
python -m uvicorn main:app --reload
```

Run python client.py in another terminal. On an idle local machine, elapsed time should be closer to 0.5 seconds than 1 second; exact timing is not guaranteed.

Multi-file examples: split at the filename comments. Do not paste non-Python files into a Python module.

# Change one thing at a time

Suggested time: 20-35 minutes. Attempt the tasks before reading the sketch.

## Task 09.1

In a disposable copy, replace `await asyncio.sleep(seconds)` with `time.sleep(seconds)`, importing `time`. Compare elapsed time.

## Task 09.2

Restore the awaitable sleep.

## Task 09.3

Explain why a long pure-Python loop inside `async def` would still delay the event loop.

## Hint

The blocking experiment is intentionally incorrect. Do not retain it as the recommended implementation.

## Solution sketch

```
# If an unavoidable synchronous I/O function must be called:
from starlette.concurrency import run_in_threadpool

# Inside an async handler, where blocking_lookup is defined:
# result = await run_in_threadpool(blocking_lookup, item_id)
# This is an integration pattern, not a standalone example.
```

## Exit criteria

- You distinguish waiting from computation.
- The final lab keeps the awaitable sleep.
- You do not infer production throughput from this toy timing.

# Call another API reliably

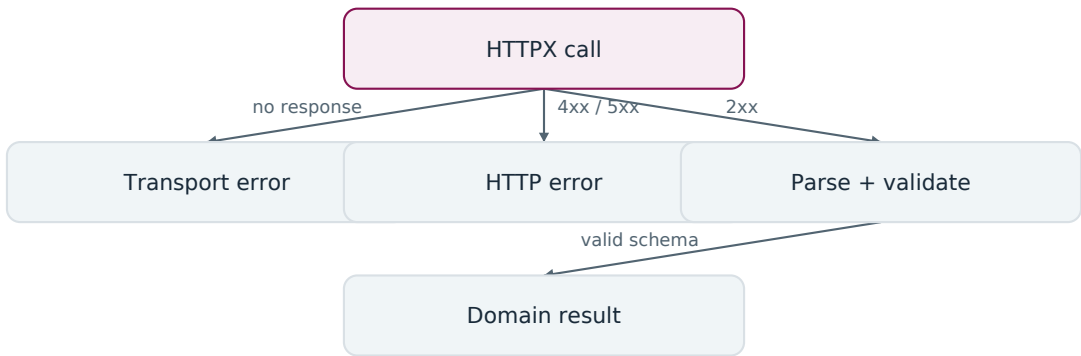
**Handle transport errors, HTTP errors and malformed payloads separately.**

Calling an upstream API introduces an independent failure boundary. A connection failure, an HTTP 503 and a 200 response containing unexpected JSON are different failures. Raising for status handles HTTP errors; it does not validate the payload schema.

Use a Client as a context-managed resource for connection reuse. Set timeouts deliberately. HTTPX distinguishes connect, read, write and pool timeouts; these are operation or inactivity limits, not a single guaranteed wall-clock deadline for the whole business operation.

Retries must be bounded and appropriate to the operation. A read-only GET may be retried for selected transient failures; do not blindly repeat a POST after a timeout because the write might already have committed. Honor Retry-After where applicable and use backoff with jitter for real traffic.

The lab maps failures into a domain exception and leaves the retry decision to the caller. The deterministic mock avoids external APIs and lets you inject one failure at a time.



| Failure               | Detection               | Typical handling                   |
|-----------------------|-------------------------|------------------------------------|
| Network timeout       | TimeoutException        | Budgeted retry if safe             |
| Bad status            | HTTPStatusError         | Classify by status                 |
| Invalid JSON/schema   | ValueError / validation | Treat as upstream contract failure |
| Unknown write outcome | POST timed out          | Reconcile or use idempotency key   |

References: [02], [14], [15]. Linked reading appears at the end.

# Read, run, inspect

## FILES main.py

```
import httpx
from pydantic import BaseModel, Field

class Price(BaseModel):
    cents: int = Field(ge=0, strict=True)

class UpstreamUnavailable(Exception):
    pass

class PriceClient:
    def __init__(self, client: httpx.Client) -> None:
        self._client = client

    def fetch(self, item_id: int) -> Price:
        try:
            response = self._client.get(f"/prices/{item_id}")
            response.raise_for_status()
            return Price.model_validate(response.json())
        except (httpx.RequestError, httpx.HTTPStatusError,
                ValueError) as exc:
            raise UpstreamUnavailable("Price lookup failed") from exc

def mock_server(request: httpx.Request) -> httpx.Response:
    return httpx.Response(200, json={"cents": 125})

def main() -> None:
    timeout = httpx.Timeout(5.0, connect=1.0)
    with httpx.Client(
        base_url="https://prices.example.test",
        timeout=timeout,
        transport=httpx.MockTransport(mock_server),
    ) as client:
        print(PriceClient(client).fetch(1).cents)

if __name__ == "__main__":
    main()
```

## Run from this lab folder

```
python main.py
```

Expect 125. The wrapper receives its HTTP client, so the caller owns creation, configuration and closing.

The next page changes only this local example. No earlier lab service is required.

# Change one thing at a time

Suggested time: 20-35 minutes. Attempt the tasks before reading the sketch.

## Task 10.1

Change the mock to status 503. Confirm UpstreamUnavailable is raised.

## Task 10.2

Return 200 with {"cents":-1}; confirm that schema failure is also translated.

## Task 10.3

Make the mock raise `httpx.ReadTimeout("slow", request=request)`. Write down why this mock verifies error handling but not elapsed timeout enforcement.

## Hint

Use `try/except UpstreamUnavailable` in a small test or `pytest.raises`. Do not add real network calls.

## Solution sketch

```
def timeout_server(request: httpx.Request) -> httpx.Response:
    raise httpx.ReadTimeout("slow", request=request)

# In a test, build the client with:
# transport=httpx.MockTransport(timeout_server)
# Then assert PriceClient(client).fetch(1) raises
# UpstreamUnavailable. Keep the original exception as __cause__.
```

## Exit criteria

- Success, bad status, bad schema and timeout are distinguishable causes.
- No unbounded retry loop is introduced.
- A mock timeout is an injected exception, not a timed network test.

# Authenticate and authorize

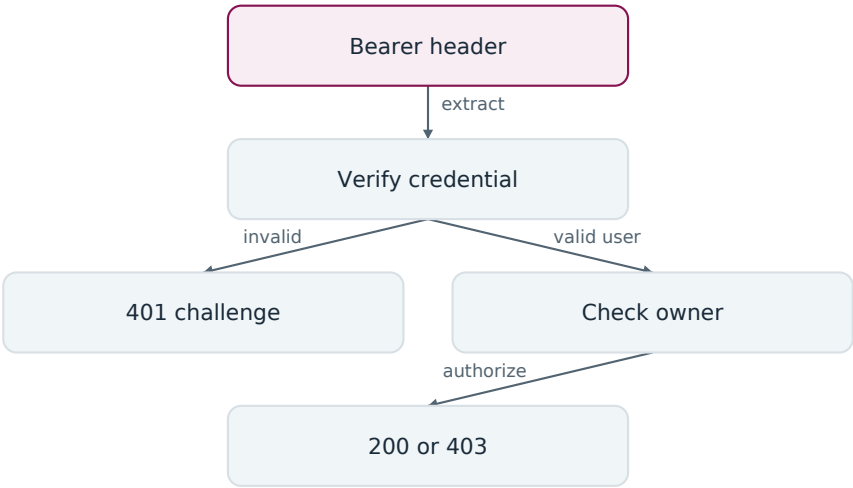
**Check both identity and access to the requested object.**

Authentication asks who is calling; authorization asks what that caller may do. Possessing a valid token does not grant access to every resource. Check ownership or permissions for each protected object.

Bearer credentials travel in the Authorization header. HTTPS protects them in transit. A JWT is a token format, not encryption and not proof of trust by itself: signature, issuer, audience and expiry checks are needed under the chosen identity system.

This lab intentionally uses a fixed, public demo token to isolate dependency wiring and ownership checks. It is not a login system or a production token verifier. Real deployments should use a maintained authentication system and a configured secret or key source.

Missing or invalid credentials receive 401 with a Bearer challenge. A known caller without access receives 403 in this example. Some systems return 404 to avoid revealing existence; choose and document a consistent policy.



| Concern             | Example                                |
|---------------------|----------------------------------------|
| Authentication      | Token maps to alice                    |
| Authorization       | Only alice can read alice-owned report |
| Secret handling     | No credentials in logs or URL          |
| Production boundary | Replace the demo verifier              |

References: [16], [17]. Linked reading appears at the end.

# Read, run, inspect

**FILES** main.py

```
import secrets
from typing import Annotated
from fastapi import Depends, FastAPI, HTTPException
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer

app = FastAPI()
bearer = HTTPBearer(auto_error=False)
Credential = Annotated[
    HTTPAuthorizationCredentials | None, Depends(bearer)
]

def current_user(credentials: Credential) -> str:
    # Public demo credential: replace the verifier in production.
    if credentials is None or not secrets.compare_digest(
        credentials.credentials, "demo-alice-token"
    ):
        raise HTTPException(
            401, detail="Invalid credentials",
            headers={"WWW-Authenticate": "Bearer"},
        )
    return "alice"

@app.get("/reports/{owner}")
def read_report(
    owner: str, user: Annotated[str, Depends(current_user)],
) -> dict[str, str]:
    if owner != user:
        raise HTTPException(403, detail="Access denied")
    return {"owner": owner, "report": "demo"}
```

## Run from this lab folder

```
python -m uvicorn main:app --reload
```

GET /reports/alice with Authorization: Bearer demo-alice-token returns 200. The same credential at /reports/bob returns 403.

The next page changes only this local example. No earlier lab service is required.

# Change one thing at a time

Suggested time: 20-35 minutes. Attempt the tasks before reading the sketch.

## Task 11.1

Call without a token; inspect both status and WWW-Authenticate.

## Task 11.2

Call with a wrong token, then the correct token for bob.

## Task 11.3

Explain why putting user=alice in a query parameter is not an authentication mechanism.

## Hint

With curl, add -H "Authorization: Bearer demo-alice-token". Treat only the fixed demo string as safe to type into this lab.

## Solution sketch

```
# TestClient assertions, using this module's app:
with TestClient(app) as client:
    response = client.get("/reports/alice")
    assert response.status_code == 401
    assert response.headers["www-authenticate"] == "Bearer"
    response = client.get(
        "/reports/bob",
        headers={"Authorization": "Bearer demo-alice-token"},
    )
    assert response.status_code == 403
# Add: from fastapi.testclient import TestClient
```

## Exit criteria

- Missing and wrong tokens return 401; wrong owner returns 403.
- You can name the exact demo-only part to replace.

# Paginate and evolve a contract

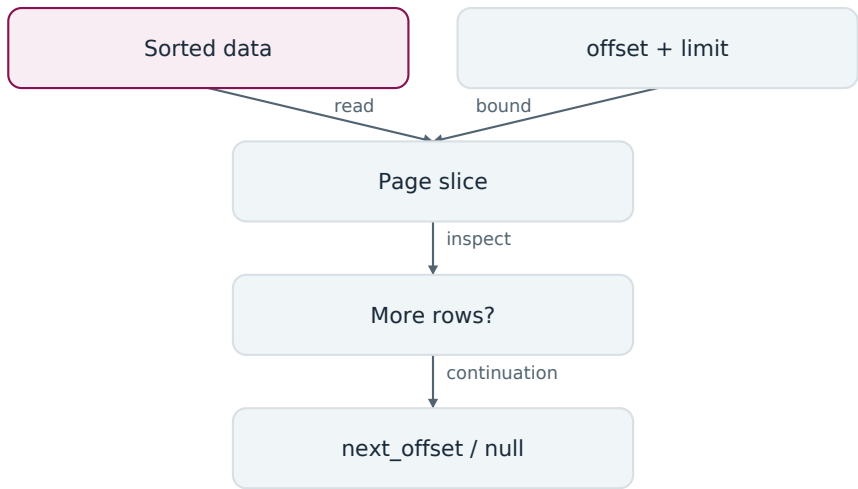
**Bound list responses and make changes predictable for callers.**

A collection endpoint should not return unbounded data. Offset pagination uses a starting index and a limit; it is easy to learn but can skip or repeat items when earlier rows change between requests. Always define a deterministic ordering.

Cursor pagination carries a continuation marker, often based on a stable ordering key such as (created\_at, id). It can avoid deep offsets, but a cursor must be validated and should not expose an unsafe query or become an authorization bypass.

A response envelope makes list metadata explicit. This lab returns items and next\_offset. An empty page is a successful result, not a missing resource. The dataset is immutable, so page behavior is deterministic.

Contract evolution includes field types, requiredness, defaults, errors and ordering, not just URLs. Renaming a response field breaks existing callers. An optional additive field is often compatible, but strict clients may still reject it. Version deliberately and test representative clients.



| Request          | IDs returned | next_offset |
|------------------|--------------|-------------|
| offset=0&limit=2 | 1, 2         | 2           |
| offset=2&limit=2 | 3, 4         | 4           |
| offset=4&limit=2 | 5            | null        |
| offset=9&limit=2 | none         | null        |

References: [18], [19]. Linked reading appears at the end.

# Read, run, inspect

**FILES** main.py

```
from typing import Annotated
from fastapi import FastAPI, Query
from pydantic import BaseModel

app = FastAPI()
ITEM_IDS = (1, 2, 3, 4, 5)

class PageResponse(BaseModel):
    items: list[int]
    next_offset: int | None

@app.get("/items", response_model=PageResponse)
def list_items(
    offset: Annotated[int, Query(ge=0)] = 0,
    limit: Annotated[int, Query(ge=1, le=3)] = 2,
) -> PageResponse:
    page = list(ITEM_IDS[offset:offset + limit])
    next_offset = offset + len(page)
    return PageResponse(
        items=page,
        next_offset=next_offset if next_offset < len(ITEM_IDS) else None,
    )
```

## Run from this lab folder

```
python -m uvicorn main:app --reload
```

GET /items?offset=0&limit=2 returns {"items":[1,2],"next\_offset":2}. Follow the continuation until null.

The next page changes only this local example. No earlier lab service is required.

# Change one thing at a time

Suggested time: 20-35 minutes. Attempt the tasks before reading the sketch.

## Task 12.1

Fetch every ID by following `next_offset`, never assuming a page is full.

## Task 12.2

Test `limit=0`, `limit=4` and `offset=-1`.

## Task 12.3

Add an optional response metadata field `total: int | None = None`; discuss the cost if `total` required a large database count.

## Hint

Stop on `next_offset` is `None`. An empty list is data, not an exception.

## Solution sketch

```
# Client-side loop with a running local server:
import httpx

with httpx.Client(base_url="http://127.0.0.1:8000") as client:
    offset = 0
    collected = []
    while True:
        response = client.get("/items", params={"offset": offset})
        response.raise_for_status()
        page = response.json()
        collected.extend(page["items"])
        if page["next_offset"] is None:
            break
        offset = page["next_offset"]
    assert collected == [1, 2, 3, 4, 5]
```

## Exit criteria

- Each ID is collected once for this fixed dataset.
- Invalid bounds return 422; an out-of-range offset returns 200.
- You can explain the limitation under concurrent inserts.

# Observe requests and understand CORS

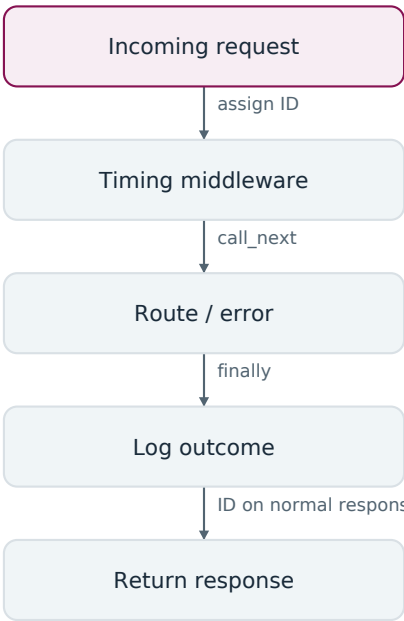
**Add request correlation and inspect browser access rules.**

Logs explain individual events; metrics aggregate behavior; traces connect work across services. Start with method, path, status, duration and a request identifier. Avoid logging bodies, tokens and sensitive query strings by default.

Middleware wraps request processing, which makes it useful for timing and correlation. It should not replace object-level authorization. The example generates a server-side ID instead of trusting an arbitrary caller-supplied value.

CORS is a browser-enforced cross-origin access policy. An origin includes scheme, host and port. Allowing a frontend origin permits its browser JavaScript to read eligible responses; it does not authenticate requests or block curl clients.

The lab logs normal responses and exceptions and configures one allowed local frontend origin. Unhandled exceptions are re-raised; this small middleware does not promise a correlation header on every framework-generated 500 response.



| Signal        | Answers                                    |
|---------------|--------------------------------------------|
| Request ID    | Which logs belong to this request?         |
| Duration      | How long did application processing take?  |
| Status counts | How frequently are requests failing?       |
| CORS header   | May this browser origin read the response? |

References: [20], [21]. Linked reading appears at the end.

# Read, run, inspect

## FILES main.py

```
import logging
from time import perf_counter
from uuid import uuid4
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from starlette.middleware.base import RequestResponseEndpoint
from starlette.responses import Response

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("api")
app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_methods=["GET"],
    allow_headers=["Authorization"],
)

@app.middleware("http")
async def observe(
    request: Request, call_next: RequestResponseEndpoint,
) -> Response:
    request_id = uuid4().hex
    start = perf_counter()
    status = 500
    try:
        response = await call_next(request)
        status = response.status_code
        response.headers["X-Request-ID"] = request_id
        return response
    except Exception:
        logger.exception("request_failed id=%s", request_id)
        raise
    finally:
        logger.info("id=%s method=%s path=%s status=%s ms=%.1f",
                    request_id, request.method, request.url.path,
                    status, (perf_counter() - start) * 1000)

@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}
```

## Run from this lab folder

```
python -m uvicorn main:app --reload
```

`curl -i http://127.0.0.1:8000/health` shows X-Request-ID. Add `-H "Origin: http://localhost:3000"` to inspect Access-Control-Allow-Origin.

The next page changes only this local example. No earlier lab service is required.

# Change one thing at a time

Suggested time: 20-35 minutes. Attempt the tasks before reading the sketch.

## Task 13.1

Make two calls and verify the request IDs differ.

## Task 13.2

Compare allowed Origin `http://localhost:3000` with `http://localhost:3001`.

## Task 13.3

Explain why curl can still read the response for the disallowed origin.

## Hint

For a normal GET, a disallowed origin typically receives a response without an allow-origin header; the browser decides whether JavaScript can read it.

## Solution sketch

```
# Add to CORSMiddleware if frontend JavaScript must read
# the correlation response header:
# expose_headers=["X-Request-ID"]

# Terminal preflight probe (remove the leading #):
# curl -i -X OPTIONS http://127.0.0.1:8000/health \
#   -H "Origin: http://localhost:3000" \
#   -H "Access-Control-Request-Method: GET"
```

## Exit criteria

- Log entries and returned IDs can be matched.
- Only the configured origin receives an allow-origin header.
- You distinguish `allow_headers` from `expose_headers`.

# Persist one resource with SQLite

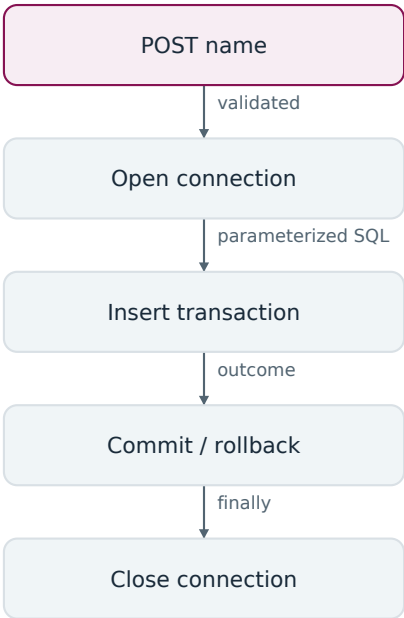
**Learn transaction and connection boundaries without an ORM.**

An in-memory dictionary belongs to one process and disappears on restart. Multiple worker processes do not automatically share it. A database gives data an independent lifetime; it also introduces transactions, constraints and contention.

Parameterized SQL separates data values from SQL syntax. Never build a query by interpolating user input. A unique database constraint is the authoritative check against duplicates; a prior SELECT alone cannot prevent a concurrent race.

This lab creates, uses and closes a connection inside one synchronous operation. A transaction context commits on success and rolls back on exception, but does not itself close a sqlite3 connection. The outer closing context handles that.

Run `init_db()` before serving. This is explicit lab setup, not a production migration system. SQLite is useful here because no database server is needed. Concurrent writes still have limits; use a deliberate database design when scaling.



| Boundary             | Owner                      |
|----------------------|----------------------------|
| Input constraints    | Pydantic request model     |
| Duplicate prevention | UNIQUE database constraint |
| Commit / rollback    | Transaction context        |
| Connection closing   | contextlib.closing         |

References: [22]. Linked reading appears at the end.

# Read, run, inspect

## FILES main.py

```
from contextlib import closing
import sqlite3
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

DB_PATH = "items.db"
app = FastAPI()

class ItemCreate(BaseModel):
    name: str = Field(min_length=1, max_length=80)

def init_db() -> None:
    with closing(sqlite3.connect(DB_PATH)) as conn, conn:
        conn.execute("CREATE TABLE IF NOT EXISTS items ("
                      "id INTEGER PRIMARY KEY, name TEXT UNIQUE NOT NULL)")

def insert_item(name: str) -> int:
    with closing(sqlite3.connect(DB_PATH)) as conn, conn:
        cursor = conn.execute("INSERT INTO items(name) VALUES (?)", (name,))
        return int(cursor.lastrowid)

@app.post("/items", status_code=201)
def create_item(payload: ItemCreate) -> dict[str, int | str]:
    try:
        item_id = insert_item(payload.name)
    except sqlite3.IntegrityError as exc:
        raise HTTPException(409, detail="Name already exists") from exc
    return {"id": item_id, "name": payload.name}

if __name__ == "__main__":
    init_db()
```

## Run from this lab folder

```
python main.py
python -m uvicorn main:app --reload
```

POST /items with {"name": "Pen"} returns 201. Repeating it returns 409. Repeating it after a server restart still returns 409.

The next page changes only this local example. No earlier lab service is required.

# Change one thing at a time

Suggested time: 20-35 minutes. Attempt the tasks before reading the sketch.

## Task 14.1

Create a new name and verify it persists after a restart.

## Task 14.2

Submit a name containing an apostrophe, such as Alice's pen.

## Task 14.3

Add a pure database read function using a parameterized SELECT. Return None when no row exists.

## Hint

Keep the connection entirely inside the operation. In this schema, valid inserts can violate the name uniqueness constraint; a larger schema needs more precise IntegrityError mapping.

## Solution sketch

```
def find_name(item_id: int) -> str | None:
    with closing(sqlite3.connect(DB_PATH)) as conn:
        row = conn.execute(
            "SELECT name FROM items WHERE id = ?", (item_id,)
        ).fetchone()
        return None if row is None else str(row[0])
```

## Exit criteria

- Repeated names conflict, including after restart.
- An apostrophe is stored as data.
- To reset only this lab, stop the server, remove items.db and rerun setup.

# Manage application lifetime and configuration

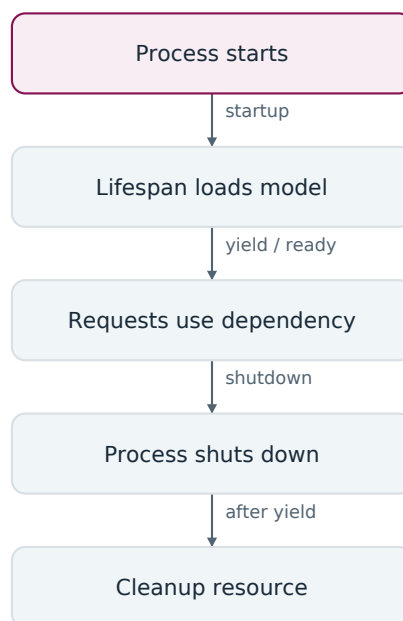
**Load a shared resource once per process and expose it through a dependency.**

Some resources should exist for the application lifetime: a connection pool, an HTTP client or an ML model. Lifespan defines startup work before yield and cleanup after yield. It makes ownership explicit instead of hiding expensive work inside every request.

The example uses a tiny fake predictor so the API lesson needs no model download, GPU or training pipeline. The same lifetime pattern can hold an actual model. Each worker process loads its own copy, so worker count can multiply memory use.

Configuration should be read at a deliberate boundary and validated early. This lab reads `MODEL_VERSION` at startup with a harmless default. Secrets require an appropriate secret source and must not appear in source code or logs.

A real predictor may be expensive or unsafe under concurrent access. The synchronous route here is fine for the pure, small fake. Loading once does not establish thread safety, throughput, batching or a GPU scheduling strategy.



| Scope               | Example            | Important property                   |
|---------------------|--------------------|--------------------------------------|
| Request             | Transaction        | Short-lived, isolated                |
| Application process | Model or HTTP pool | Shared within one worker             |
| External service    | Database           | Can be shared across workers         |
| Environment setting | MODEL_VERSION      | Read at intentional startup boundary |

References: [23], [24]. Linked reading appears at the end.

# Read, run, inspect

## FILES main.py

```
from collections.abc import AsyncIterator
from contextlib import asynccontextmanager
import os
from typing import Annotated
from fastapi import Depends, FastAPI, Request
from pydantic import BaseModel, Field

class Predictor:
    def predict(self, value: float) -> float:
        return value * 2

    def close(self) -> None:
        pass # No external resource in this fake.

@asynccontextmanager
async def lifespan(app: FastAPI) -> AsyncIterator[None]:
    app.state.model_version = os.getenv("MODEL_VERSION", "demo-v1")
    app.state.predictor = Predictor()
    try:
        yield
    finally:
        app.state.predictor.close()

app = FastAPI(lifespan=lifespan)

class PredictRequest(BaseModel):
    value: float = Field(allow_inf_nan=False)

def get_predictor(request: Request) -> Predictor:
    return request.app.state.predictor

@app.post("/predict")
def predict(
    payload: PredictRequest, request: Request,
    predictor: Annotated[Predictor, Depends(get_predictor)],
) -> dict[str, float | str]:
    return {"prediction": predictor.predict(payload.value),
            "model_version": request.app.state.model_version}
```

## Run from this lab folder

```
MODEL_VERSION=demo-v2 python -m uvicorn main:app --reload
```

POST /predict with {"value":3} returns prediction=6.0 and model\_version=demo-v2. The command uses a macOS/Linux environment-variable prefix.

The next page changes only this local example. No earlier lab service is required.

# Change one thing at a time

Suggested time: 20-35 minutes. Attempt the tasks before reading the sketch.

## Task 15.1

Test `/predict` with `TestClient(app)` as a context manager.

## Task 15.2

Add `GET /ready` that returns 200 only when the predictor exists, otherwise 503.

## Task 15.3

Explain why four worker processes can require approximately four model copies.

## Hint

Read readiness through `request.app.state`. A liveness check only asks whether the process is alive; readiness asks whether it can serve required work.

## Solution sketch

```
from fastapi import HTTPException

@app.get("/ready")
def ready(request: Request) -> dict[str, str]:
    if getattr(request.app.state, "predictor", None) is None:
        raise HTTPException(503, detail="Model not ready")
    return {"status": "ready"}
```

## Exit criteria

- The context-managed test sees initialized state.
- The model is not constructed inside `predict()`.
- The readiness example is a basic local check, not ongoing dependency health.

# Run the API in a container

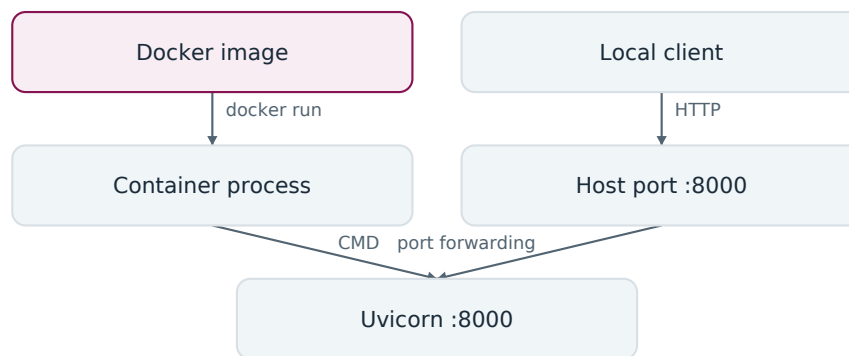
## Distinguish an application, a server process and its packaging.

An API contract describes how callers interact. Uvicorn serves the Python application. Docker packages the program, dependencies and startup command into an image. Running that image starts a container; packaging does not change the API semantics.

Inside a container, bind the server to 0.0.0.0 so forwarded traffic can reach it. The host port mapping below listens only on 127.0.0.1 for a local exercise. EXPOSE documents a port; it does not publish that port on the host.

Development reload watches files and restarts processes; omit it for ordinary production serving. A container restart also restarts the app lifecycle. External persistence, readiness, resource limits, HTTPS termination and monitoring need explicit operational decisions.

This final lab packages only a health endpoint. There is no cloud account, database or deployment platform. Pin and test dependencies for reproducibility; the provided command records exact installed versions in your isolated course environment.



| Concept                | Meaning                                   |
|------------------------|-------------------------------------------|
| Image                  | Packaged filesystem and launch metadata   |
| Container              | Running instance with process isolation   |
| 0.0.0.0 in container   | Listen on container interfaces            |
| -p 127.0.0.1:8000:8000 | Map local host port to container port     |
| HTTPS termination      | Usually configured at a proxy or platform |

References: [25]. Linked reading appears at the end.

# Read, run, inspect

## FILES main.py + Dockerfile + .dockerignore

```
# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}

# Dockerfile (separate file; not Python)
FROM python:3.12-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
COPY requirements.lock.txt .
RUN pip install --no-cache-dir -r requirements.lock.txt
RUN useradd --create-home appuser
COPY main.py .
USER appuser
EXPOSE 8000
CMD ["python", "-m", "uvicorn", "main:app", "--host", "0.0.0.0"]

# .dockerignore (separate file)
.venv/
__pycache__/.
.env
.git/
*.db
```

## Run from this lab folder

```
python -m pip freeze > requirements.lock.txt
docker build -t api-lab .
docker run --rm -p 127.0.0.1:8000:8000 api-lab
```

With Docker installed and running, call `curl -i http://127.0.0.1:8000/health`. Expect 200. Stop any earlier Uvicorn process using port 8000 first.

Multi-file examples: split at the filename comments. Do not paste non-Python files into a Python module.

# Change one thing at a time

Suggested time: 20-35 minutes. Attempt the tasks before reading the sketch.

## Task 16.1

Build and run the image, then call /health.

## Task 16.2

Map host port 8001 to container port 8000 and call the new URL.

## Task 16.3

Explain where you would terminate HTTPS and where persistent data would live for a real service.

## Hint

The left port is on the host; the right port is inside the container. Docker must be installed separately; this lab is optional if it is unavailable.

## Solution sketch

```
# Terminal:
docker run --rm -p 127.0.0.1:8001:8000 api-lab

# In another terminal:
curl -i http://127.0.0.1:8001/health

# For repeatable real builds, also choose a reviewed base-image
# digest and lock dependencies for the target Python/platform.
```

## Exit criteria

- Changing the host port needs no Python code change.
- The service runs as a non-root user and without --reload.
- A successful local container call is not proof of production readiness.

# HTTP decisions at a glance

| Code                  | Use in this course                          | Response expectation                         |
|-----------------------|---------------------------------------------|----------------------------------------------|
| 200                   | Successful read or computation              | Documented representation                    |
| 201                   | Resource created                            | Created representation; Location when useful |
| 204                   | Successful operation with no content        | No body                                      |
| 400                   | General malformed/invalid request semantics | Document the exact application policy        |
| 401                   | Missing/invalid credentials                 | Authentication challenge                     |
| 403                   | Authenticated but forbidden                 | No protected data                            |
| 404                   | Resource or route unavailable               | Absence or non-disclosure policy             |
| 405                   | Method unavailable for target               | Correct the requested method                 |
| 409                   | Conflict with current state                 | Resolvable conflict information              |
| 422                   | FastAPI request validation / lab null rule  | Input error details                          |
| 429                   | Rate limit exceeded                         | Respect retry guidance when supplied         |
| 500                   | Unexpected server failure                   | Generic public error; internal diagnostics   |
| 502 /<br>503 /<br>504 | Gateway/upstream or availability failures   | Failure-specific policy; bounded retries     |

## Error is a boundary, not one exception type

Validate the transport outcome, then the HTTP status, then the response representation. Apply a total operation budget when downstream calls or retries can accumulate. A connection/read timeout on one attempt is only one part of that budget.

## Before adding another class

Ask: what dependency or state does it own, who creates it, who closes it, and what test becomes easier? If the answers are unclear, start with a typed function in a coherent module.

# From learning code to a real service

| Concern        | Useful starting pattern            | What must be decided for real use              |
|----------------|------------------------------------|------------------------------------------------|
| Contract       | Input and output schemas           | Compatibility, bounds, documented errors       |
| Business logic | Pure functions or injected service | Rules, authorization and failure semantics     |
| Storage        | Parameterized SQLite operations    | Migrations, contention, backups, transactions  |
| Upstream API   | Injected HTTPX client              | Timeout budget, safe retries, payload checks   |
| Authentication | Dependency boundary                | Real verifier, key rotation, expiry, ownership |
| Model lifetime | Lifespan-owned instance            | Memory per worker, concurrency, batching       |
| Operations     | Logs, health, local container      | HTTPS, metrics, readiness, resource limits     |

## A short debugging sequence

1. Reproduce one failing request with its exact method, path, headers and body. 2. Read status and response body. 3. Match a request ID to logs. 4. Check whether failure occurred in routing, validation, authorization, business logic or a collaborator. 5. Add one regression test at that boundary.

## Optional synthesis exercise

If you want one final integration exercise, combine only Modules 03, 07, 08 and 15 into POST /predict. Keep the fake model, validate input, inject the predictor and test success plus invalid input. Stop there; a database, frontend, login system and cloud account are unnecessary for this goal.

## Verification of this edition

The Python worked examples were checked on Python 3.12 with the pinned baseline. Checks covered HTTP contract behavior, dependency replacement, authentication outcomes, pagination, CORS, SQLite insertion/conflict and lifespan initialization. Module 08 tests were run with pytest. The Docker recipe is provided for local execution; a container build was not run in this authoring environment.

The async wait example illustrates concurrency, not a benchmark. The upstream mock illustrates failures, not real network timeout enforcement. Diagrams are conceptual application flows rather than packet traces.

# Questions 01-05

Answer in your own words or short Python. Assume the stated context; include necessary imports when writing a complete file. Use a separate notebook if you need more room. Answers are in the later solutions section.

## O01 | Module 01 | Theory

**Describe the request and response for reading item 7. Include method, path, one request header, status, one response header and a JSON body.**

---

---

## O02 | Module 01 | Diagnosis

**A server returns 200 with the text "maintenance". Why can `response.json()` still fail, and where should the client handle that?**

---

---

## O03 | Module 01 | Quick code

**Write a context-managed HTTPX GET to `https://api.example.test/items` with `query limit=2`, a 3-second timeout and status checking. No live request is required.**

---

---

## O04 | Module 02 | Theory

**Explain the separate responsibilities of FastAPI, Uvicorn and `main:app`.**

---

---

## O05 | Module 02 | Quick code

**Write GET `/ping` with a typed handler returning `{"message": "pong"}`. Assume `app` is a FastAPI instance.**

---

---

# Questions 06-10

## O06 | Module 02 | Diagnosis

**GET /health works but POST /health returns 405. Explain the cause and why adding a POST route is not automatically the right fix.**

---

---

## O07 | Module 03 | Quick code

**Define a Pydantic request model with a strict integer quantity from 1 to 20 and no unknown fields.**

---

---

## O08 | Module 03 | Theory

**For /items/12?include\_stock=true, identify a path value and a query value. Where should a structured item-create payload go?**

---

---

## O09 | Module 03 | Diagnosis

**A quantity annotated int accepts a compatible string in a non-strict model. Is the annotation being ignored? Explain how to reject this input.**

---

---

## O10 | Module 04 | Theory

**Explain safe and idempotent using GET, PUT and DELETE. Must repeated DELETE responses have identical status codes?**

---

---

# Questions 11-15

## O11 | Module 04 | Quick code

Given payload is a PATCH model, get only supplied fields and reject any explicitly supplied null with HTTP 422.

---

---

## O12 | Module 04 | Design

Design the success responses for creating an item and deleting an item without a response body. Include status codes and a useful creation header.

---

---

## O13 | Module 05 | Theory

Why should a pure inventory service raise a domain exception rather than FastAPI HTTPException?

---

---

## O14 | Module 05 | Design

Propose an error envelope for an unavailable item. Explain which part a client should branch on.

---

---

## O15 | Module 05 | Diagnosis

An endpoint catches Exception and returns [] with 200. Name two concrete harms and show a better direction.

---

---

# Questions 16-20

## O16 | Module 06 | Quick code

Write a pure `subtotal(quantity, unit_cents)` function with annotations. Reject quantity below 1 and negative prices.

---

---

## O17 | Module 06 | Design

Choose a function or class for (a) cents-to-dollars formatting and (b) a client that holds an injected HTTPX client. Justify both.

---

---

## O18 | Module 06 | Diagnosis

`routes.py` imports `main.app`, and `main.py` imports `routes.router`. Identify the risk and redraw the import direction in words.

---

---

## O19 | Module 07 | Theory

What does a repository Protocol buy a route? What does it not guarantee?

---

---

## O20 | Module 07 | Quick code

Write a `get_service` provider and an Annotated dependency alias for a `Service` class that has a no-argument constructor.

---

---

# Questions 21-25

## O21 | Module 07 | Diagnosis

A test sets `app.dependency_overrides` and the next test unexpectedly gets fake data. Explain and fix the leak.

---

---

## O22 | Module 08 | Quick code

Write a `TestClient` test that sends `{"value":0}` to `/double` and asserts validation failure. Assume a client fixture exists.

---

---

## O23 | Module 08 | Design

List four useful API test cases for quantity constrained to strict integers from 1 to 10. Explain what each catches.

---

---

## O24 | Module 08 | Theory

Why can a `TestClient` suite pass while the deployed service is unreachable?

---

---

## O25 | Module 09 | Theory

Explain concurrency versus parallelism using two requests waiting for a remote service.

---

---

# Questions 26-30

## O26 | Module 09 | Diagnosis

An async route directly calls `time.sleep(1)`. Explain the impact and give two possible fixes.

---

---

## O27 | Module 09 | Design

A prediction takes two seconds of CPU work. Why is adding async insufficient? Name two capacity strategies.

---

---

## O28 | Module 10 | Theory

Distinguish an HTTP timeout, a returned 503 and invalid JSON in a 200 response.

---

---

## O29 | Module 10 | Design

A create-payment POST times out after the server may have committed. Why is a blind retry unsafe, and what contract could help?

---

---

## O30 | Module 10 | Quick code

Write a `MockTransport` handler that raises an `HTTPX ReadTimeout` associated with its incoming request.

---

---

# Questions 31-35

## O31 | Module 11 | Theory

**Distinguish authentication and object-level authorization using Alice reading Bob's report.**

---

---

## O32 | Module 11 | Design

**State the response for missing Bearer credentials and for a valid caller who is forbidden from an object under this course policy.**

---

---

## O33 | Module 11 | Diagnosis

**A developer decodes a JWT payload and trusts its `user_id`. Explain why decoding alone is insufficient.**

---

---

## O34 | Module 12 | Quick code

**For an already-sorted list `rows`, `offset` and `limit`, compute a `page` and a `next_offset` that is `None` when exhausted.**

---

---

## O35 | Module 12 | Theory

**How can offset pagination duplicate an item when a new row is inserted before the next page?**

---

---

# Questions 36-40

## O36 | Module 12 | Design

An API changes `price_cents` from integer to string. Explain the compatibility risk and a safer migration.

---

---

## O37 | Module 13 | Theory

Explain why CORS is neither authentication nor a server-side firewall.

---

---

## O38 | Module 13 | Design

Choose five fields for a request completion log and name two categories to exclude by default.

---

---

## O39 | Module 13 | Quick code

Write a short finally block that logs elapsed milliseconds using `perf_counter` and an existing start variable.

---

---

## O40 | Module 14 | Quick code

Write a parameterized SQLite SELECT of name by `item_id` using an existing conn.

---

---

# Questions 41-45

## O41 | Module 14 | Theory

**Why is checking for a duplicate name with SELECT before INSERT insufficient under concurrent requests?**

---

---

## O42 | Module 14 | Diagnosis

**Code uses with sqlite3.connect(path) as conn and assumes the connection is closed afterward. Correct the assumption.**

---

---

## O43 | Module 15 | Theory

**Why should a large model load in lifespan instead of inside each prediction handler?**

---

---

## O44 | Module 15 | Quick code

**Sketch an async lifespan context manager that acquires resource, yields, and always closes resource. Assume synchronous acquire and close for a cheap fake.**

---

---

## O45 | Module 15 | Design

**Distinguish liveness from readiness for an API whose model fails to load.**

---

---

# Questions 46-50

## O46 | Module 16 | Theory

**Explain image, container and port mapping using host 8001 and container 8000.**

---

---

## O47 | Module 16 | Quick code

**Write a Uvicorn launch command for main:app inside a container listening on port 8000, without development reload.**

---

---

## O48 | Module 16 | Diagnosis

**A container was built successfully, but curl to the host fails. Give three targeted checks.**

---

---

## O49 | Module 10 | Design

**Design a bounded retry policy for a read-only upstream GET. State retryable cases, limits and observability.**

---

---

## O50 | Module 15 | Design

**Sketch the responsibilities for a small /predict API with schema validation, a reusable model, an upstream lookup and tests. Keep the design minimal.**

---

---

# Questions 001-005

Choose one best answer for each question. All items use the course contract unless stated otherwise. The answer key includes a short explanation and a module number for review.

**M001. Which part of an HTTP response indicates the outcome category?**

- A. Python function name
- B. Status code
- C. Host port
- D. Query string

**M002. What does HTTPX json={"x": 1} do for a request?**

- A. Creates a database row locally
- B. Encodes the mapping as a JSON body
- C. Parses the response body
- D. Adds x to the URL path

**M003. Which header describes the actual media type of a response body?**

- A. Location
- B. Authorization
- C. Content-Type
- D. Accept

**M004. What is limit=2 in /items?limit=2?**

- A. A query parameter
- B. A response header
- C. A path parameter named items
- D. A TCP port

**M005. A 200 response contains invalid JSON. Which operation can fail?**

- A. response.json()
- B. Reading the URL string
- C. Reading response.status\_code
- D. Accessing the request method

# Questions 006-010

**M006. What does MockTransport remove from a basic HTTPX lab?**

- A. The need for a real network server
- B. The need to validate payloads
- C. All HTTP status codes
- D. The need to close clients

**M007. In main:app, what does main identify?**

- A. The domain name
- B. The importable Python module
- C. The TCP protocol
- D. The virtual environment

**M008. Which component listens for HTTP when using Uvicorn with FastAPI?**

- A. A Pydantic model
- B. pytest
- C. requirements.lock.txt
- D. Uvicorn

**M009. Which pair uniquely distinguishes these API operations?**

- A. Function length and filename
- B. Port and response message
- C. HTTP method and path
- D. JSON key order and indentation

**M010. What is /openapi.json normally used for in FastAPI?**

- A. The Python package lock
- B. The machine-readable API schema
- C. An authentication secret
- D. A database backup

# Questions 011-015

**M011. What is `--reload` primarily intended for?**

- A. Making all handlers asynchronous
- B. Enabling HTTPS automatically
- C. Sharing memory between workers
- D. Restarting the development server after edits

**M012. `GET /health` exists; `POST /health` is unregistered. What status is normally expected?**

- A. 201
- B. 204
- C. 405
- D. 409

**M013. Which constraint accepts integers from 1 through 10 inclusive?**

- A. `Field(ge=1, le=10)`
- B. `Field(ge=0, le=11)`
- C. `Field(min_length=1)`
- D. `Field(gt=1, lt=10)`

**M014. What does `ConfigDict(extra="forbid")` do?**

- A. Encrypts all fields
- B. Makes every field optional
- C. Disables all validation
- D. Rejects unknown model input fields

**M015. What normally happens before a FastAPI handler receives an invalid request body?**

- A. The handler silently gets an empty model
- B. The database is always updated
- C. The request becomes GET
- D. The framework returns a validation error

# Questions 016-020

**M016. Which input is rejected by a strict int body field?**

- A. The JSON number 2
- B. The JSON number 10
- C. The JSON string "2"
- D. The JSON number 1

**M017. Where does a structured create-item payload normally belong?**

- A. The server port
- B. The POST request body
- C. The response Location header
- D. The status line

**M018. What does a valid request schema fail to prove?**

- A. That required fields were present
- B. That local field constraints passed
- C. That parsing completed
- D. That the caller is authorized

**M019. Which method is safe by HTTP semantics?**

- A. POST
- B. DELETE
- C. GET
- D. PATCH

**M020. What does idempotent mean?**

- A. Repeating a request has the same intended effect as sending it once
- B. The operation has no side effects at all
- C. Every response has identical text
- D. The operation always succeeds

# Questions 021-025

**M021. Which method conventionally replaces a representation at a known target?**

- A. PUT
- B. HEAD
- C. GET
- D. OPTIONS

**M022. Which PATCH operation is not idempotent without extra safeguards?**

- A. Set quantity to exactly 5
- B. Set active to false
- C. Set name to Pen
- D. Increment a counter by one

**M023. What does `model_dump(exclude_unset=True)` preserve for a PATCH?**

- A. Only fields whose values are truthy
- B. Only fields equal to defaults
- C. Only required fields
- D. Only fields explicitly supplied by the caller

**M024. What body should a 204 deletion response contain?**

- A. The text success
- B. The deleted object
- C. No response body
- D. An empty JSON object

**M025. A first DELETE returns 204 and a second 404. Can the operation still be idempotent?**

- A. Yes, if the intended final state is the same
- B. Only if the second request is GET
- C. No, the statuses must match
- D. Only if both responses have JSON

# Questions 026-030

**M026. What should clients prefer when branching on an error envelope?**

- A. A stable machine-readable error code
- B. The exact English message
- C. The server traceback
- D. The order of JSON keys

**M027. Which status fits a known duplicate-name conflict in this course?**

- A. 409
- B. 301
- C. 204
- D. 200

**M028. Where should a domain exception normally become an HTTP response?**

- A. At the web boundary
- B. Inside every pure calculation
- C. In the Dockerfile
- D. Inside the JSON encoder only

**M029. Why is returning 200 with [] after any exception misleading?**

- A. It disables TLS
- B. It always deletes data
- C. It forces a POST request
- D. It makes failures look like successful empty results

**M030. Which information should not be returned in a public unexpected-error response?**

- A. A stack trace containing internal details
- B. An appropriate 500-class status
- C. A generic failure message
- D. A request correlation identifier

# Questions 031-035

**M031. A server constructs an invalid response model. Who normally owns the defect?**

- A. The server implementation
- B. The DNS resolver
- C. The caller merely because it sent a request
- D. The browser CORS policy

**M032. Which design fits a deterministic cents calculation without dependencies?**

- A. An HTTP middleware
- B. A pure function
- C. A database table
- D. A mandatory singleton class

**M033. Which is the strongest reason to use a client wrapper class?**

- A. Every function must be in a class
- B. Classes automatically validate JSON
- C. Classes eliminate network timeouts
- D. It holds an injected collaborator used by related operations

**M034. Which module should normally assemble the app and include routers?**

- A. The SQL query string
- B. The request schema class
- C. The composition module such as main.py
- D. The pure pricing module

**M035. Which dependency direction is preferable?**

- A. Schemas create the Uvicorn process
- B. Routes depend on domain functions
- C. Every module imports every other module
- D. Domain functions import main.app

# Questions 036-040

**M036. What is APIRouter used for?**

- A. Managing SQLite transactions
- B. Encrypting API tokens
- C. Grouping related route registrations
- D. Running Docker containers

**M037. When should another architectural layer be introduced?**

- A. When a concrete responsibility or substitution need justifies it
- B. For every line of logic
- C. To ensure every function has three wrappers
- D. Only after adding Kubernetes

**M038. What does Depends(get\_repository) receive?**

- A. The result of calling get\_repository immediately
- B. A provider callable
- C. A TCP socket number
- D. The text of a SQL query

**M039. What does a Protocol describe in the repository example?**

- A. The number of worker processes
- B. The Docker image base
- C. The operations the consumer requires
- D. The HTTP wire encoding

**M040. Must a fake explicitly inherit from a Protocol to satisfy it structurally?**

- A. Only for GET routes
- B. Only when using SQLite
- C. Yes, always
- D. No, matching the required behavior can suffice

# Questions 041-045

**M041. Default dependency caching in FastAPI is normally scoped to what?**

- A. All worker processes together
- B. The lifetime of a Docker image
- C. One request
- D. All deployments forever

**M042. What should a dependency override mapping value normally be?**

- A. A URL query parameter
- B. An exception traceback
- C. A callable that supplies the replacement
- D. An arbitrary JSON response string

**M043. What prevents a fake override leaking into another test?**

- A. Rename the HTTP method
- B. Clear it in teardown or use a clean app fixture
- C. Change the server port
- D. Add extra JSON fields

**M044. What does an API contract test primarily check?**

- A. Private variable names
- B. Observable status, body and headers
- C. Exact source indentation
- D. The author's editor settings

**M045. Why use TestClient as a context manager for a lifespan app?**

- A. To build a Docker image
- B. To force internet access
- C. To disable schema validation
- D. To run startup and shutdown around the test

# Questions 046-050

**M046. What does an in-process TestClient test not prove?**

- A. That the real host port is reachable
- B. That the handler returned a JSON value
- C. That a tested status equals 200
- D. That validation rejected a tested input

**M047. Which input pair targets valid boundaries of ge=1, le=100?**

- A. 50 and 51
- B. 1 and 100
- C. 2 and 99
- D. 0 and 101

**M048. Which assertion is usually least brittle for a standard input error?**

- A. Assert request elapsed time equals exactly 1 ms
- B. Assert the entire human error sentence forever
- C. Assert status 422 and relevant structured fields
- D. Assert all internal stack frame names

**M049. What does pytest parameterization help express?**

- A. Implicit user authentication
- B. Several input cases for the same behavioral rule
- C. Automatic production deployment
- D. A shared database across machines

**M050. What does awaitable I/O let an event loop do while waiting?**

- A. Automatically add CPU cores
- B. Turn JSON into SQL
- C. Eliminate all network latency
- D. Run other ready tasks

# Questions 051-055

**M051. What happens when `time.sleep` is called directly in `async def`?**

- A. It becomes nonblocking automatically
- B. It creates a separate process
- C. It validates the response schema
- D. It blocks the event-loop thread during the sleep

**M052. Where do ordinary FastAPI `def` route handlers generally run?**

- A. Always on a GPU
- B. In the browser
- C. A thread pool
- D. Inside the database engine

**M053. A `def` helper is called directly inside an `async` handler. Is it automatically offloaded?**

- A. No
- B. Only if it returns a dictionary
- C. Only if its name starts with `get`
- D. Yes, every `def` call is offloaded

**M054. What does changing a heavy CPU function to `async def` guarantee?**

- A. Automatic GPU batching
- B. A lower memory footprint
- C. No automatic CPU speedup
- D. Parallel execution on every core

**M055. What is a suitable way to invoke unavoidable blocking I/O from an `async` route?**

- A. Add a JSON response model only
- B. Call it directly and assume it yields
- C. Rename the function with an `async` prefix
- D. Explicitly offload it to a thread pool

# Questions 056-060

**M056. Which method raises for an unsuccessful HTTP status in the course client?**

- A. `request.encode_status()`
- B. `client.close_status()`
- C. `response.json()`
- D. `response.raise_for_status()`

**M057. What does a read timeout generally bound in HTTPX?**

- A. The lifetime of the Python process
- B. Total business-operation duration under all retries
- C. Model loading time before the request
- D. Waiting for response data during a read operation

**M058. A POST timed out after possibly committing. What is the key risk of retrying?**

- A. Making the response model optional
- B. Duplicating the write
- C. Changing the HTTP version
- D. Removing the Authorization header automatically

**M059. Which is the best general retry approach for a read-only GET?**

- A. Bound selected transient retries with a time budget and backoff
- B. Retry malformed payloads thousands of times
- C. Retry every failure forever
- D. Retry all 401 responses without changing credentials

**M060. Who closes the HTTPX client injected into PriceClient in Module 10?**

- A. Every fetch call by default
- B. The Price model
- C. The remote server
- D. The caller that created the context-managed client

# Questions 061-065

**M061. Which mock response exercises payload validation after HTTP success?**

- A. 200 with {"cents": -1}
- B. A connection refusal
- C. A missing hostname
- D. 503 with an empty body

**M062. A MockTransport raises ReadTimeout immediately. What has the test established?**

- A. TLS verification is enabled on a real connection
- B. A real five-second timeout elapsed
- C. The server is unreachable over DNS
- D. The code handles that exception

**M063. Which question is authorization?**

- A. Which port does Uvicorn use?
- B. May Alice read this particular report?
- C. Who does the token identify?
- D. What JSON encoding is used?

**M064. Which response accompanies missing credentials under the course policy?**

- A. 301 to the database
- B. 200 with an empty list
- C. 401 with WWW-Authenticate: Bearer
- D. 204 with a token body

**M065. A valid caller is denied access to a known object under the lab policy. Which status?**

- A. 204
- B. 201
- C. 422
- D. 403

# Questions 066-070

**M066. Why is decoding a JWT insufficient?**

- A. Decoded JSON is always encrypted
- B. It does not itself verify authenticity and required claims
- C. JWT payloads cannot contain user IDs
- D. JWTs only work with SQLite

**M067. Where should a Bearer token normally be sent?**

- A. A Docker image label
- B. A public URL query string
- C. Authorization header over HTTPS
- D. A response status code

**M068. Which part of the authentication lab is intentionally not production-ready?**

- A. The fixed public demo-token verifier
- B. The concept of an ownership check
- C. Returning JSON for a successful request
- D. Using a dependency parameter

**M069. Why should a collection endpoint bound limit?**

- A. To guarantee immutable data
- B. To disable client authentication
- C. To control response size and work per request
- D. To avoid using HTTP status codes

**M070. What is required for predictable pagination?**

- A. A defined deterministic ordering
- B. Different field types on each page
- C. Random ordering on each request
- D. A new server port for every page

# Questions 071-075

**M071. What should the client do when next\_offset is null in the lab?**

- A. Restart from offset zero
- B. Stop fetching pages
- C. Treat it as HTTP 500
- D. Double the page size forever

**M072. What normally represents an empty successful list page?**

- A. 500 with a traceback
- B. 404 for the entire collection every time
- C. 200 with an empty items list
- D. 201 with a new resource

**M073. Why can offset pagination repeat data after an insertion?**

- A. Earlier insertion shifts row positions
- B. The URL loses its host
- C. JSON cannot encode integer IDs
- D. HTTP GET is never idempotent

**M074. Which change is clearly risky for existing numeric clients?**

- A. Renaming a private local variable
- B. Reformatting the Dockerfile
- C. Fixing an internal comment
- D. Changing price\_cents from integer to string

**M075. What makes two browser URLs different origins?**

- A. Only a difference in query order
- B. A difference in scheme, host or port
- C. Only a difference in URL path
- D. A different JSON body

# Questions 076-080

**M076. A disallowed-origin curl request still reads the body. Why?**

- A. curl does not enforce browser CORS policy
- B. All credentials have been disabled
- C. The server must be compromised
- D. CORS requires SQLite

**M077. Which data should generally be excluded from ordinary request logs?**

- A. Server-generated request IDs
- B. Request durations
- C. Bearer tokens
- D. Response status codes

**M078. What does a request ID primarily support?**

- A. Automatically retrying writes
- B. Correlating events for one request
- C. Encrypting the response
- D. Replacing all metrics

**M079. What allows browser JavaScript to read X-Request-ID on an eligible cross-origin response?**

- A. Remove all CORS headers
- B. Expose that response header in CORS configuration
- C. Change the response to status 204
- D. Put it only in allow\_methods

**M080. Why log in finally around request processing?**

- A. To record completion timing even when an exception occurs
- B. To force every response to 200
- C. To disable startup events
- D. To suppress every exception

# Questions 081-085

**M081. Which SQL pattern correctly treats an item name as data?**

- A. String concatenation around user input
- B. Executing the name as a query
- C. INSERT INTO items(name) VALUES (?) with a parameter tuple
- D. An f-string containing the raw name

**M082. What reliably enforces unique names against concurrent inserts?**

- A. A browser CORS rule
- B. A response model alone
- C. Only a preceding SELECT
- D. A database UNIQUE constraint

**M083. What does a sqlite3 connection transaction context not do by itself?**

- A. Provide context-manager entry and exit
- B. Roll back an unsuccessful transaction
- C. Close the connection
- D. Commit a successful transaction

**M084. Why does a module-level Python dictionary not provide durable storage?**

- A. It cannot contain strings
- B. It belongs to one process and is lost on restart
- C. It always rejects duplicate keys with 409
- D. It automatically becomes JSON

**M085. What happens when two separate worker processes each use their own dictionary?**

- A. They automatically share every mutation
- B. They prevent all race conditions
- C. They create a distributed transaction
- D. They have separate in-memory state

# Questions 086-090

**M086. Why use a fresh local SQLite file for an isolated persistence lab?**

- A. It avoids an external database server while exercising persistence
- B. It encrypts all names automatically
- C. It removes the need for transactions
- D. It proves unlimited concurrent-write capacity

**M087. Where should a reusable model usually be acquired in the shown pattern?**

- A. Inside each response serializer
- B. Inside every prediction request
- C. Application lifespan startup
- D. In a CORS allowlist

**M088. What does code after yield in the lifespan context normally handle?**

- A. Shutdown cleanup
- B. Docker image building
- C. Every path parameter conversion
- D. Browser preflight parsing

**M089. Four workers each load a model. What is a likely memory implication?**

- A. The model disappears after the first request
- B. Async eliminates model memory
- C. Approximately four process-local model copies
- D. Exactly one shared copy is guaranteed

**M090. What does readiness ask?**

- A. Is the source code formatted?
- B. Has the user passed the exam?
- C. Is the Docker image public?
- D. Can this instance serve required work?

# Questions 091-095

**M091. What does loading a model once fail to establish?**

- A. Thread safety and sufficient serving capacity
- B. A startup acquisition boundary
- C. A place to perform cleanup
- D. Reuse within that process

**M092. When is MODEL\_VERSION read in the course example?**

- A. Only by the browser
- B. During application startup
- C. On every JSON key access
- D. When the PDF is opened

**M093. What does EXPOSE 8000 do in a Dockerfile?**

- A. Publishes host port 8000 automatically
- B. Documents the intended container port
- C. Enables HTTPS
- D. Starts Uvicorn by itself

**M094. Why bind Uvicorn to 0.0.0.0 inside the container?**

- A. To make request bodies optional
- B. To accept traffic on container interfaces
- C. To encrypt all connections
- D. To choose every host port

**M095. What does -p 127.0.0.1:8001:8000 map?**

- A. HTTPS to SQL
- B. Python 8001 to Python 8000
- C. Host loopback port 8001 to container port 8000
- D. Container port 8001 to every host port

# Questions 096-100

**M096. Which command flag should normally be omitted for production serving?**

- A. --host
- B. --port
- C. The module:app import string
- D. --reload

**M097. Why record exact dependency versions after testing?**

- A. To remove all OS differences
- B. To create an authentication system
- C. To make the tested environment easier to reproduce
- D. To guarantee all future releases are compatible

**M098. What does a successful docker build alone prove?**

- A. HTTPS is correctly configured
- B. The API is reachable on the host
- C. The production database is durable
- D. The image build completed

**M099. Which status most directly communicates that a new resource was created?**

- A. 200
- B. 201
- C. 404
- D. 304

**M100. The process responds to a basic ping but its required model is unavailable. Which interpretation fits?**

- A. Its API has become idempotent
- B. It may be live but not ready
- C. It must be fully ready
- D. It is necessarily a CORS error

# Model answers 01-05

These are concise model answers, not the only acceptable wording. For code, equivalent typed implementations are valid. Snippets assume the imports and context named in the question or its module. Score with the 0-4 rubric on page 2.

## O01 | Module 01

GET /items/7; Accept: application/json. A successful response can be 200, Content-Type: application/json, and {"id":7,"name":"Pen"}. Explain which side produces each element.

## O02 | Module 01

HTTP success does not imply a JSON body. Decode and validate at the client boundary; translate the decoding failure into an upstream contract error instead of treating it as useful data.

## O03 | Module 01

```
with httpx.Client(timeout=3.0) as client:
    response = client.get("https://api.example.test/items", params={"limit": 2})
    response.raise_for_status()
```

## O04 | Module 02

FastAPI defines and runs application behavior; Uvicorn accepts network traffic and invokes the ASGI app. main:app is the Python module and object Uvicorn imports.

## O05 | Module 02

```
@app.get("/ping")
def ping() -> dict[str, str]:
    return {"message": "pong"}
```

# Model answers 06-10

## O06 | Module 02

The path exists but that method is not registered. If the intended operation is a read, the client should use GET. Add POST only if the contract needs a separate POST operation.

## O07 | Module 03

```
class OrderRequest(BaseModel):  
    model_config = ConfigDict(extra="forbid")  
    quantity: int = Field(ge=1, le=20, strict=True)
```

## O08 | Module 03

12 is the path value; include\_stock=true is a query parameter. A structured create payload normally goes in the JSON request body of POST /items.

## O09 | Module 03

No: parsing/coercion can produce an integer. Apply Field(strict=True) or a suitable strict model policy, and test the exact input types required by the contract.

## O10 | Module 04

Safe requests do not ask for state changes. PUT and DELETE are not safe but are idempotent by semantics. DELETE may succeed once and then return 404 while leaving the same intended final state.

# Model answers 11-15

## O11 | Module 04

```
changes = payload.model_dump(exclude_unset=True)
if any(value is None for value in changes.values()):
    raise HTTPException(422, detail="Null values are not allowed")
```

## O12 | Module 04

Create: 201 with the created representation and, where applicable, Location: /items/{id}. Delete: 204 with no response body. Document how missing IDs are handled.

## O13 | Module 05

A domain exception communicates a business failure without coupling the service to HTTP. A web adapter maps it to a status and public error shape; other adapters can map it differently.

## O14 | Module 05

Example: {"error":{"code":"item\_not\_found","message":"Item not found"}} with 404. Clients should branch on the stable code, not the wording of message.

## O15 | Module 05

It makes failed operations look like successful empty results and hides defects from monitoring/callers. Catch expected exceptions narrowly and map them; let unexpected failures be logged and become server errors.

# Model answers 16-20

## O16 | Module 06

```
def subtotal(quantity: int, unit_cents: int) -> int:
    if quantity < 1 or unit_cents < 0:
        raise ValueError("Invalid input")
    return quantity * unit_cents
```

## O17 | Module 06

A pure function fits formatting with no owned state. A class fits a client wrapper that retains a configured dependency and exposes related operations. Neither choice requires a generic base class.

## O18 | Module 06

This creates circular imports and hidden global access. `main.py` should create the app and include `routes.router`; `routes.py` imports domain functions or providers, while domain modules never import `main.py`.

## O19 | Module 07

It describes the operations the route requires and allows structural test doubles. It does not create storage, guarantee runtime correctness, manage lifetimes or automatically enforce every type at runtime.

## O20 | Module 07

```
def get_service() -> Service:
    return Service()
ServiceDep = Annotated[Service, Depends(get_service)]
```

# Model answers 21-25

## O21 | Module 07

The same app retains the override mapping. Clear it in finally or use fresh app fixtures with proper teardown. Overrides are test setup, not persistent configuration.

## O22 | Module 08

```
def test_rejects_zero(client: TestClient) -> None:
    response = client.post("/double", json={"value": 0})
    assert response.status_code == 422
```

## O23 | Module 08

1 and 10 test the valid boundaries; 0 or 11 tests range rejection; "2" tests strict type rejection. Additional useful cases include missing input and malformed JSON.

## O24 | Module 08

It runs inside the Python process and bypasses real DNS, listening ports, host mapping, firewall, proxy and TLS. Add a separate smoke check against the running service.

## O25 | Module 09

Concurrent requests can overlap their waits and resume as results arrive. Parallelism means simultaneous execution. Awaiting I/O supports concurrency without proving parallel CPU execution.

# Model answers 26-30

## O26 | Module 09

It blocks the event loop. Use `await asyncio.sleep` for an intentional asynchronous delay, or offload a genuinely blocking I/O call / use a synchronous `def` handler as appropriate.

## O27 | Module 09

The calculation still occupies execution resources and may block the loop. Consider a process-based worker/job service or a dedicated model-serving service; measure resource use and concurrency before selecting capacity.

## O28 | Module 10

A timeout is a transport/operation failure; 503 is an actual HTTP response signaling service unavailability; invalid JSON is a representation failure after an HTTP response. Handle them at distinct boundaries.

## O29 | Module 10

A retry may create a second payment. Use a server-supported idempotency key with stored outcomes and payload consistency checks, or reconcile the operation status before repeating the write.

## O30 | Module 10

```
def slow(request: httpx.Request) -> httpx.Response:
    raise httpx.ReadTimeout("slow", request=request)
```

# Model answers 31-35

## O31 | Module 11

Authentication establishes Alice as the caller. Authorization checks whether Alice may access Bob's specific report. A valid credential alone does not satisfy the second check.

## O32 | Module 11

Missing/invalid credentials: 401 with WWW-Authenticate: Bearer. Authenticated but forbidden: 403. A documented non-disclosure policy may use 404 for the second case.

## O33 | Module 11

Decoding reads data but does not establish authenticity. Validate signature, trusted algorithm/key, issuer, audience and expiry according to the identity system, then enforce authorization.

## O34 | Module 12

```
page = rows[offset:offset + limit]
end = offset + len(page)
next_offset = end if end < len(rows) else None
```

## O35 | Module 12

The insertion shifts later row positions. The next numeric offset can point back to an item already read. Stable ordering helps determinism but does not freeze a changing dataset.

# Model answers 36-40

## O36 | Module 12

Existing clients may perform numeric operations or reject strings. Keep the old contract while adding a new field/version, document a transition and validate representative clients before removing the old field.

## O37 | Module 13

CORS controls whether browser JavaScript may read certain cross-origin responses. It does not establish caller identity and non-browser clients are not bound by the browser policy.

## O38 | Module 13

Request ID, method, path or route template, status and duration. Exclude credentials and sensitive bodies/query values by default. Avoid high-cardinality IDs as aggregate metric labels.

## O39 | Module 13

```
finally:
    elapsed_ms = (perf_counter() - start) * 1000
    logger.info("duration_ms=%.1f", elapsed_ms)
```

## O40 | Module 14

```
row = conn.execute(
    "SELECT name FROM items WHERE id = ?", (item_id,)
).fetchone()
```

# Model answers 41-45

## O41 | Module 14

Two requests may both observe absence and then insert. A database uniqueness constraint enforces the rule atomically; catch and map the relevant constraint violation.

## O42 | Module 14

The connection context manages commit or rollback, not closing. Add `contextlib.closing` or explicitly close in finally, and keep the transaction boundary intentional.

## O43 | Module 15

Lifespan loads it before readiness and allows reuse per process, avoiding repeated initialization latency and resource use. It also gives cleanup a defined place.

## O44 | Module 15

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    resource = acquire()
    try:
        yield
    finally:
        resource.close()
```

## O45 | Module 15

Liveness asks whether the process is functioning. Readiness asks whether it can serve required work. A process can be alive but unready; a startup failure may prevent it from serving at all.

# Model answers 46-50

## O46 | Module 16

An image packages the software. A container is its running instance. Mapping 127.0.0.1:8001:8000 forwards local host port 8001 to the service listening on container port 8000.

## O47 | Module 16

```
python -m uvicorn main:app --host 0.0.0.0 --port 8000
```

## O48 | Module 16

Verify the container is running and inspect logs; confirm Uvicorn binds 0.0.0.0 on the expected container port; confirm the published host port and the URL. A build alone does not start serving.

## O49 | Module 10

For example, retry selected connection/timeouts and transient 502/503/504 at most twice within an overall budget; use capped backoff with jitter and honor Retry-After as applicable. Log attempt counts and terminal causes. Do not retry every 4xx or malformed payload.

## O50 | Module 15

Schemas validate input/output; a route translates HTTP; an injected prediction service coordinates a lifespan-owned model and a bounded-time upstream client. Unit-test pure logic, fake the upstream in API tests, and smoke-test the running service. Add modules only at those concrete seams.

# Answer key 001-020

| Item | Key | Why this answer is correct                                                           |
|------|-----|--------------------------------------------------------------------------------------|
| M001 | B   | The status code communicates the HTTP outcome. [Module 01]                           |
| M002 | B   | json= performs request JSON serialization. [Module 01]                               |
| M003 | C   | Content-Type describes the body sent; Accept expresses a preference. [Module 01]     |
| M004 | A   | The portion after ? contains query parameters. [Module 01]                           |
| M005 | A   | HTTP status success and JSON parsing are independent. [Module 01]                    |
| M006 | A   | A mock supplies responses in process; contract checks still matter. [Module 01]      |
| M007 | B   | Uvicorn imports the module and finds its app attribute. [Module 02]                  |
| M008 | D   | Uvicorn is the ASGI server process. [Module 02]                                      |
| M009 | C   | GET /items and POST /items are different operations. [Module 02]                     |
| M010 | B   | OpenAPI describes the API contract. [Module 02]                                      |
| M011 | D   | Reload is a development convenience. [Module 02]                                     |
| M012 | C   | 405 means the method is not allowed for that target. [Module 02]                     |
| M013 | A   | ge and le include their boundaries. [Module 03]                                      |
| M014 | D   | Unknown fields become validation errors. [Module 03]                                 |
| M015 | D   | Boundary validation rejects the request before normal handler execution. [Module 03] |
| M016 | C   | Strict integer validation does not coerce strings. [Module 03]                       |
| M017 | B   | Structured creation input belongs in the request body. [Module 03]                   |
| M018 | D   | Authorization requires caller and resource permission checks. [Module 03]            |
| M019 | C   | GET asks to retrieve, not to change application state. [Module 04]                   |
| M020 | A   | Idempotency concerns intended server-side effect. [Module 04]                        |

# Answer key 021-040

| Item | Key | Why this answer is correct                                                        |
|------|-----|-----------------------------------------------------------------------------------|
| M021 | A   | PUT expresses replacement semantics at the target. [Module 04]                    |
| M022 | D   | Repeating an increment adds another increment. [Module 04]                        |
| M023 | D   | It distinguishes omitted fields from explicitly supplied fields. [Module 04]      |
| M024 | C   | 204 means No Content; do not send JSON with it. [Module 04]                       |
| M025 | A   | Different responses can reflect the same final deletion state. [Module 04]        |
| M026 | A   | Stable codes decouple clients from wording changes. [Module 05]                   |
| M027 | A   | 409 represents a conflict with the current state. [Module 05]                     |
| M028 | A   | The adapter owns protocol-specific translation. [Module 05]                       |
| M029 | D   | Callers and monitoring may interpret failure as success. [Module 05]              |
| M030 | A   | Detailed traces belong in controlled internal diagnostics. [Module 05]            |
| M031 | A   | Invalid server output is not an ordinary client input error. [Module 05]          |
| M032 | B   | A function expresses stateless computation clearly. [Module 06]                   |
| M033 | D   | A class can give related operations a meaningful dependency lifetime. [Module 06] |
| M034 | C   | Composition belongs at the application assembly boundary. [Module 06]             |
| M035 | B   | Keep core behavior independent of the HTTP app instance. [Module 06]              |
| M036 | C   | Routers help organize related HTTP operations. [Module 06]                        |
| M037 | A   | Layers are useful when they clarify real boundaries. [Module 06]                  |
| M038 | B   | FastAPI invokes the provider as part of dependency resolution. [Module 07]        |
| M039 | C   | The interface supports structural collaborators and fakes. [Module 07]            |
| M040 | D   | Protocols support structural typing. [Module 07]                                  |

# Answer key 041-060

| Item | Key | Why this answer is correct                                                          |
|------|-----|-------------------------------------------------------------------------------------|
| M041 | C   | Request caching is not an application-wide singleton. [Module 07]                   |
| M042 | C   | Overrides replace provider callables with other callables. [Module 07]              |
| M043 | B   | Control test state and clean up dependency overrides. [Module 07]                   |
| M044 | B   | Tests should target behavior callers depend on. [Module 08]                         |
| M045 | D   | The context manages application lifespan events. [Module 08]                        |
| M046 | A   | Network deployment needs its own smoke check. [Module 08]                           |
| M047 | B   | The inclusive minimum and maximum are boundary cases. [Module 08]                   |
| M048 | C   | Avoid overfitting to volatile wording or timing. [Module 08]                        |
| M049 | B   | Parameterization makes boundary and invalid-input cases concise. [Module 08]        |
| M050 | D   | Cooperative suspension permits concurrent progress. [Module 09]                     |
| M051 | D   | A synchronous blocking call does not become awaitable by context. [Module 09]       |
| M052 | C   | FastAPI offloads ordinary synchronous handlers to a thread pool. [Module 09]        |
| M053 | A   | Offloading handlers does not automatically offload direct helper calls. [Module 09] |
| M054 | C   | Async syntax does not remove computation cost. [Module 09]                          |
| M055 | D   | Use an intentional offload boundary or a suitable synchronous handler. [Module 09]  |
| M056 | D   | Status checking is separate from body decoding. [Module 10]                         |
| M057 | D   | Timeout phases do not create a universal wall-clock deadline. [Module 10]           |
| M058 | B   | A timeout can leave the write outcome unknown. [Module 10]                          |
| M059 | A   | Retry policy must be bounded and failure-aware. [Module 10]                         |
| M060 | D   | Resource ownership stays with the creator in this pattern. [Module 10]              |

# Answer key 061-080

| Item | Key | Why this answer is correct                                                            |
|------|-----|---------------------------------------------------------------------------------------|
| M061 | A   | The negative value violates the schema despite status 200. [Module 10]                |
| M062 | D   | Injected failures verify handling, not real network timing. [Module 10]               |
| M063 | B   | Authorization determines permission for an action or object. [Module 11]              |
| M064 | C   | 401 challenges the caller to authenticate. [Module 11]                                |
| M065 | D   | 403 represents forbidden access for the authenticated caller. [Module 11]             |
| M066 | B   | Trust requires cryptographic and claim validation. [Module 11]                        |
| M067 | C   | The Authorization header is the standard Bearer credential location. [Module 11]      |
| M068 | A   | Replace the toy verifier with a real identity system. [Module 11]                     |
| M069 | C   | Unbounded list responses can exhaust resources. [Module 12]                           |
| M070 | A   | A consistent order is necessary even though mutable data can still shift. [Module 12] |
| M071 | B   | Null marks the end of this pagination contract. [Module 12]                           |
| M072 | C   | An empty query result is still a successful list operation. [Module 12]               |
| M073 | A   | Numeric offsets refer to positions in a changing ordering. [Module 12]                |
| M074 | D   | Changing a public field type can break client behavior. [Module 12]                   |
| M075 | B   | Origin consists of scheme, host and port. [Module 13]                                 |
| M076 | A   | CORS is a browser access policy, not a universal client restriction. [Module 13]      |
| M077 | C   | Credentials are sensitive and unnecessary for routine completion logs. [Module 13]    |
| M078 | B   | An ID connects related diagnostic entries. [Module 13]                                |
| M079 | B   | expose_headers governs readable non-safelisted response headers. [Module 13]          |
| M080 | A   | finally executes for normal and exceptional exits. [Module 13]                        |

# Answer key 081-100

| Item | Key | Why this answer is correct                                                               |
|------|-----|------------------------------------------------------------------------------------------|
| M081 | C   | Bound parameters separate values from SQL syntax. [Module 14]                            |
| M082 | D   | The database constraint is authoritative across racing writes. [Module 14]               |
| M083 | C   | Use closing or explicit close for connection lifetime. [Module 14]                       |
| M084 | B   | Process memory has a different lifetime from persisted data. [Module 14]                 |
| M085 | D   | Separate process memory is not a shared datastore. [Module 14]                           |
| M086 | A   | SQLite keeps the setup narrow without removing database concepts. [Module 14]            |
| M087 | C   | Lifespan gives model acquisition an intentional per-process lifetime. [Module 15]        |
| M088 | A   | The post-yield section releases application resources. [Module 15]                       |
| M089 | C   | Worker processes ordinarily own separate model instances. [Module 15]                    |
| M090 | D   | Readiness controls whether the instance should receive work. [Module 15]                 |
| M091 | A   | Resource lifetime and concurrency safety are different concerns. [Module 15]             |
| M092 | B   | Configuration is read at the lifespan boundary. [Module 15]                              |
| M093 | B   | Host publication requires a mapping or platform configuration. [Module 16]               |
| M094 | B   | Forwarded traffic must reach a listening container interface. [Module 16]                |
| M095 | C   | The left address/port is on the host; the right port is in the container. [Module 16]    |
| M096 | D   | Development reload is not the ordinary production execution mode. [Module 16]            |
| M097 | C   | Version locks improve reproducibility but do not erase platform differences. [Module 16] |
| M098 | D   | Build, run, connectivity and operational correctness are separate checks. [Module 16]    |
| M099 | B   | 201 Created explicitly identifies resource creation. [Module 04]                         |
| M100 | B   | Liveness alone does not establish that required resources are available. [Module 15]     |

# Official documentation

Primary documentation checked 5 September 2026. Links support the named API behavior; the course examples, exercises and assessments are original teaching material. Documentation can change after publication.

## **[01] HTTP message anatomy**

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Messages>

## **[02] HTTPX quickstart**

<https://www.python-httpx.org/quickstart/>

## **[03] FastAPI first steps**

<https://fastapi.tiangolo.com/tutorial/first-steps/>

## **[04] Pydantic models**

<https://docs.pydantic.dev/latest/concepts/models/>

## **[05] FastAPI numeric validation**

<https://fastapi.tiangolo.com/tutorial/path-params-numeric-validations/>

## **[06] HTTP method semantics**

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods>

## **[07] FastAPI partial body updates**

<https://fastapi.tiangolo.com/tutorial/body-updates/>

## **[08] FastAPI error handling**

<https://fastapi.tiangolo.com/tutorial/handling-errors/>

## **[09] FastAPI larger applications**

<https://fastapi.tiangolo.com/tutorial/bigger-applications/>

## **[10] FastAPI dependencies**

<https://fastapi.tiangolo.com/tutorial/dependencies/>

## **[11] FastAPI test dependency overrides**

<https://fastapi.tiangolo.com/advanced/testing-dependencies/>

## **[12] FastAPI testing**

<https://fastapi.tiangolo.com/tutorial/testing/>

## **[13] FastAPI concurrency**

<https://fastapi.tiangolo.com/async/>

# Official documentation

## **[14] HTTPX timeout configuration**

<https://www.python-httpx.org/advanced/timeouts/>

## **[15] HTTPX exception hierarchy**

<https://www.python-httpx.org/exceptions/>

## **[16] FastAPI security introduction**

<https://fastapi.tiangolo.com/tutorial/security/first-steps/>

## **[17] OWASP API object-level authorization**

<https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/>

## **[18] FastAPI query parameters**

<https://fastapi.tiangolo.com/tutorial/query-params/>

## **[19] Microsoft REST API guidelines**

<https://github.com/microsoft/api-guidelines/blob/vNext/azure/Guidelines.md>

## **[20] FastAPI middleware**

<https://fastapi.tiangolo.com/tutorial/middleware/>

## **[21] FastAPI CORS**

<https://fastapi.tiangolo.com/tutorial/cors/>

## **[22] Python sqlite3**

<https://docs.python.org/3/library/sqlite3.html>

## **[23] FastAPI lifespan**

<https://fastapi.tiangolo.com/advanced/events/>

## **[24] FastAPI settings**

<https://fastapi.tiangolo.com/advanced/settings/>

## **[25] FastAPI containers**

<https://fastapi.tiangolo.com/deployment/docker/>